Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
«Решение уравнения Риккати с помощью General Projected Riccati Algorithm»

Выполнил:
Пахомов Владимир Юрьевич, студент группы N3250

_____
(подпись)

Проверил:
Ерофеев Сергей Анатольевич

_____
(отметка о выполнении)

_____
(подпись)

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение **3**

1 Описание работы алгоритма и реализации программы **4**

1.1 Условия применимости алгоритма 5

1.1.1 Стабилизируемость пары $(A,\ B)$ 5

1.1.2 Обнаруживаемость пары $(C,\ A)$ 5

1.1.3 Условие единственности решения уравнения Ляпунова 5

1.1.4 Требования к параметрам POD-базиса 6

1.2 Чтение входных данных и инициализация памяти 6

1.3 Построение проектора методом POD (функция `build_pod_basis`) 6

1.4 Итерационный цикл редукции размерности 7

1.4.1 Принцип решения уравнения Ляпунова методом Кронекеровой векторизации 8

1.5 Метод Ньютона–Клейнмана и решатель уравнений Ляпунова 10

1.6 Восстановление решения и оценка полной невязки 11

1.7 Физический смысл матриц и примеры применения 11

1.7.1 Пример 1. Стабилизация обратного маятника 11

1.7.2 Пример 2. Управление температурой в трёхзонной промышленной печи 12

1.8 Пример численного решения и проверка условий применимости 12

2 Описание используемых переменных **14**

3 Блок-схема алгоритма **17**

4 Тестирование программы **17**

5 Исходный код программы **24**

Заключение **41**

Приложение А **42**

ВВЕДЕНИЕ

Цель работы — изучение теоретических основ проекционных методов редукции моделей (Model Order Reduction) и практическая разработка программного комплекса для численного решения непрерывного алгебраического уравнения Риккати (CARE) на основе проекционного алгоритма (General Projected Riccati Algorithm).

Для достижения поставленной цели в рамках лабораторной работы решаются следующие задачи:

1. Изучение и реализация метода собственных ортогональных компонент (POD — Proper Orthogonal Decomposition) на основе траектории (снимков состояния) сопряженной системы, интегрируемой численным методом Рунге-Кутты 4-го порядка.
2. Реализация алгоритма проецирования исходной высокоразмерной системы (A, B, C) на ортонормированный POD-базис для получения редуцированной системы меньшего порядка.
3. Реализация итерационного метода Ньютона-Клейнмана для решения уравнения Риккати малой размерности.
4. Разработка алгоритма решения общих (в том числе несимметричных) уравнений Ляпунова, возникающих на каждом шаге метода Ньютона, с использованием Кронекеровой векторизации и метода Гаусса с частичным выбором ведущего элемента.
5. Проведение численных экспериментов на тестовых моделях различной структуры и оценка нормы невязки восстановленного решения в зависимости от размерности проекционного пространства (l).

1 ОПИСАНИЕ РАБОТЫ АЛГОРИТМА И РЕАЛИЗАЦИИ ПРОГРАММЫ

Программный комплекс реализован на языке C в виде консольного приложения, работающего со стандартными потоками ввода-вывода (`stdin`/`stdout`).

Рассматривается линейная непрерывная динамическая система вида:

$$\dot{x}(t) = A\,x(t) + B\,u(t), \tag{1}$$

$$y(t) = C\,x(t), \tag{2}$$

где $x(t) \in \mathbb{R}^N$ — вектор состояния, $u(t) \in \mathbb{R}^m$ — вектор управления, $y(t) \in \mathbb{R}^p$ — вектор измеримых выходов.

Задача линейно-квадратичного регулятора (LQR) состоит в нахождении закона управления $u(t) = -Kx(t)$, минимизирующего квадратичный функционал:

$$J = \int_0^\infty \left(x^T(t)\,Q\,x(t) + u^T(t)\,u(t)\right) dt, \tag{3}$$

где штрафная матрица $Q = C^TC \geq 0$ задаётся через матрицу выходов. Оптимальная матрица регулятора вычисляется как $K = B^TP$, где $P \in \mathbb{R}^{N\times N}$ — стабилизирующее решение *непрерывного алгебраического уравнения Риккати* (Continuous Algebraic Riccati Equation, CARE):

$$A^TP + PA - PBB^TP + C^TC = 0. \tag{4}$$

Матрицы системы имеют следующий физический смысл:

— $A \in \mathbb{R}^{N\times N}$ — **матрица динамики**: описывает свободную эволюцию системы (1) при $u \equiv 0$. Если все собственные значения A имеют отрицательную вещественную часть, система устойчива без управления; наличие хотя бы одного собственного значения с положительной вещественной частью означает неустойчивость;

— $B \in \mathbb{R}^{N\times m}$ — **матрица входов**: определяет, как управляющее воздействие $u(t)$ влияет на изменение каждой компоненты вектора состояния;

— $C \in \mathbb{R}^{p\times N}$ — **матрица выходов**: задаёт, какие линейные комбинации компонент вектора состояния измеряются датчиками;

— $P \in \mathbb{R}^{N\times N}$ — **искомая матрица решения** Риккати. В физически реализуемых системах P симметрична ($P = P^T$) и положительно полуопределена ($P \geq 0$). Элемент P_{ij} отражает «стоимость» совместного отклонения i-й и j-й компонент состояния — насколько дорого (в смысле расхода управления) исправить данное отклонение оптимальным образом.

При N большой размерности прямое решение уравнения (4) требует $O(N^3)$ операций и $O(N^2)$ памяти, что делает его неприемлемым для задач большой размерности. Для преодоления этого ограничения применяется *General Projected Riccati Algorithm*: исходная система редуцируется на подпространство малой размерности $l \ll N$, построенное методом POD, и уравнение Риккати решается в этом подпространстве.

1.1 Условия применимости алгоритма

Для существования стабилизирующего решения $P \geq 0$ уравнения (4) и корректной работы алгоритма необходимо выполнение следующих условий.

1.1.1 Стабилизируемость пары $(A,\, B)$

Пара (A, B) называется *стабилизируемой*, если существует матрица K такая, что замкнутая матрица $A - BK$ устойчива:

$$\exists\, K \in \mathbb{R}^{m\times N}\colon \quad \operatorname{Re}\lambda_i(A - BK) < 0, \quad i = 1, \ldots, N. \tag{5}$$

Физически это означает, что все неустойчивые моды системы доступны для управления через вход $u(t)$. Если какая-либо неустойчивая мода не зависит от B, подавить её невозможно и стабилизирующего решения не существует.

Практически проверить стабилизируемость можно по критерию Хаутуса: пара (A, B) стабилизируема тогда и только тогда, когда

$$\operatorname{rank}\begin{bmatrix} A - \lambda I & B \end{bmatrix} = N \quad \forall\, \lambda \in \mathbb{C}, \quad \operatorname{Re}\lambda \geq 0. \tag{6}$$

1.1.2 Обнаруживаемость пары $(C,\, A)$

Пара (C, A) называется *обнаруживаемой*, если существует матрица L такая, что матрица $A - LC$ устойчива:

$$\exists\, L \in \mathbb{R}^{N\times p}\colon \quad \operatorname{Re}\lambda_i(A - LC) < 0, \quad i = 1, \ldots, N. \tag{7}$$

Физически это означает, что все неустойчивые моды системы отражаются в измеримых выходах $y(t)$. Если неустойчивая мода не проявляется ни в одном датчике, её нельзя обнаружить и, следовательно, стабилизировать по выходу.

Критерий Хаутуса для обнаруживаемости:

$$\operatorname{rank}\begin{bmatrix} A - \lambda I \\ C \end{bmatrix} = N \quad \forall\, \lambda \in \mathbb{C}, \quad \operatorname{Re}\lambda \geq 0. \tag{8}$$

При выполнении условий (5) и (7) стабилизирующее решение $P \geq 0$ уравнения (4) существует и единственно.

1.1.3 Условие единственности решения уравнения Ляпунова

На каждом шаге метода Ньютона–Клейнмана решается уравнение Ляпунова относительно матрицы $A_{cl} = A_l - B_l B_l^T P_k$. Уравнение Ляпунова $A_{cl}^T X + X A_{cl} = -F$ имеет единственное решение тогда и только тогда, когда матрица A_{cl} не имеет пар собственных значений (λ_i, λ_j) таких, что $\lambda_i + \lambda_j = 0$. Достаточным условием является устойчивость A_{cl}, которая обеспечивается при выполнении условий стабилизируемости и обнаруживаемости уже начиная с

нулевого приближения.

1.1.4 Требования к параметрам POD-базиса

Для получения содержательного проекционного подпространства необходимо:

1. Конечное время T_{final} должно быть достаточно большим, чтобы траектория сопряжённой системы $\dot{x} = A^T x$ охватила все существенные направления пространства состояний. Для устойчивых систем рекомендуется $T_{final} \gtrsim 5/|\operatorname{Re} \lambda_{min}(A)|$, где λ_{min} — наименьшее по модулю собственное значение.
2. Число шагов $steps$ должно обеспечивать точность метода Рунге–Кутты 4-го порядка: рекомендуется $steps \geq 100$, чтобы шаг $\Delta t = T_{final}/steps$ был достаточно мал.
3. Заданный ранг r не должен превышать N. В программе это учтено автоматически: $r_{max} = \min(r, N)$.

1.2 Чтение входных данных и инициализация памяти

При запуске программа выполняет последовательное считывание параметров модели и матриц системы:

1. Через стандартный поток ввода считывается заголовок, содержащий размерности исходной системы (N, m, p), целевой ранг проектора r, количество шагов интегрирования $steps$ и конечное время траектории T_{final}.
2. Выполняется проверка корректности размерностей. Рабочий ранг проектора ограничивается физическим размером системы: $r_{max} = \min(r, N)$.
3. С помощью функции `xcalloc` динамически выделяется память под одномерные массивы типа `double` для хранения матриц исходной системы:
   — матрица динамики: $A \in \mathbb{R}^{N \times N}$;
   — матрица входов: $B \in \mathbb{R}^{N \times m}$;
   — матрица выходов: $C \in \mathbb{R}^{p \times N}$;

   в строчном формате (row-major).
4. Осуществляется построчное чтение элементов матриц A, B и C.

1.3 Построение проектора методом POD (функция `build_pod_basis`)

Для формирования оптимального проекционного подпространства вызывается функция `build_pod_basis`, выполняющая следующие действия:

1. Выделяется память под траекторную матрицу снимков Y размера $N \times S$, где общее число снимков $S = p \cdot (steps + 1)$.
2. Запускается цикл по выходам системы $out = 0, \ldots, p-1$. Для каждого выхода:
   — В качестве начального вектора сопряжённого состояния $x(0) \in \mathbb{R}^N$ берётся строка с индексом out матрицы выходов C, транспонированная в столбец:

$$x(0) = C^T_{out,\cdot}$$

— На протяжении $steps$ шагов с шагом $\Delta t = T_{final}/steps$ выполняется интегрирование сопряжённой системы:

$$\dot{x}(t) = A^T x(t).$$

На каждом шаге новое состояние $x(t_s)$ вычисляется классическим методом Рунге–Кутты 4-го порядка (`rk4_step_AT`).

— Вектор состояния $x(t_s)$ на каждом шаге s копируется в соответствующий столбец матрицы снимков Y.

3. Вычисляются элементы симметричной матрицы Грамиана $G \in \mathbb{R}^{S\times S}$ как скалярные произведения снимков, взвешенные по правилу трапеций:

$$G_{a,b} = \Delta t \cdot w_a w_b \cdot \langle Y_{\cdot,a},\, Y_{\cdot,b}\rangle = \Delta t \cdot w_a w_b \sum_{i=1}^{N} Y_{i,a}\, Y_{i,b},$$

где весовые коэффициенты $w_a,\ w_b$ равны $0{,}5$ для начального и конечного снимков и $1{,}0$ для промежуточных.

4. С помощью классического итерационного алгоритма вращений Якоби (`jacobi_sym_eigen`) находятся собственные значения w и собственные векторы U матрицы Грамиана G:

$$G\, U_{\cdot,k} = \lambda_k\, U_{\cdot,k}, \quad k = 1, \ldots, S.$$

5. Полученные пары $(\lambda_k,\ U_{\cdot,k})$ сортируются по убыванию собственных значений (`sort_eigs_desc`).
6. Осуществляется фильтрация: в базис включаются только компоненты с $\lambda_k > 10^{-12}$. Рассчитывается эффективный ранг r_{eff}.
7. Направления POD-мод формируются в матрице $V \in \mathbb{R}^{N\times r_{max}}$ по формуле:

$$V_{i,k} = \frac{1}{\sqrt{\lambda_k}} \sum_{j=1}^{S} Y_{i,j}\, U_{j,k}, \quad i = 1, \ldots, N, \quad k = 1, \ldots, r_{eff}.$$

8. Ввиду накопления численной погрешности при суммировании снимков, столбцы матрицы V дополнительно ортонормируются процедурой Грама–Шмидта (`orthonormalize_columns`).

1.4 Итерационный цикл редукции размерности

После построения базиса определяется фактический диапазон исследования: $r_{use} = \min(r,\ r_{eff})$. Программа запускает цикл по размерности редуцированного подпространства $l = 1, \ldots, r_{use}$. На каждом шаге l выполняются следующие операции:

1. Выделяется память под временный проектор V_l размера $N \times l$, в который копируются первые l столбцов из общего базиса V (`copy_first_columns`).

2. Вызывается функция `project_system`, вычисляющая спроецированные матрицы системы:

$$A_l = V_l^T A V_l \in \mathbb{R}^{l\times l},$$
$$B_l = V_l^T B \in \mathbb{R}^{l\times m},$$
$$C_l = C\, V_l \in \mathbb{R}^{p\times l}.$$

3. Инициализируется память под решение редуцированного уравнения Риккати $P_l \in \mathbb{R}^{l\times l}$, после чего вызывается функция `solve_care_newton` для решения:

$$A_l^T P_l + P_l A_l - P_l B_l B_l^T P_l + C_l^T C_l = 0.$$

1.4.1 Принцип решения уравнения Ляпунова методом Кронекеровой векторизации

Уравнение Ляпунова в общем виде записывается следующим образом:

$$A_{cl}^T X + X A_{cl} = -F, \tag{9}$$

где $A_{cl} \in \mathbb{R}^{n\times n}$ — заданная (не обязательно симметричная) матрица, $F \in \mathbb{R}^{n\times n}$ — известная правая часть, а $X \in \mathbb{R}^{n\times n}$ — искомая матрица. Несмотря на то что уравнение (9) записано в матричном виде, его можно свести к обычной системе линейных алгебраических уравнений (СЛАУ), применив операцию векторизации.

Операция векторизации. Для матрицы $X \in \mathbb{R}^{n\times n}$ операция $\mathrm{vec}(X)$ формирует вектор размера n^2, последовательно укладывая строки матрицы одна за другой:

$$\mathrm{vec}(X) = \begin{bmatrix} X_{1,1} & X_{1,2} & \cdots & X_{1,n} & X_{2,1} & \cdots & X_{n,n} \end{bmatrix}^T \in \mathbb{R}^{n^2}.$$

Кронекерово произведение. Для матриц $A \in \mathbb{R}^{p\times q}$ и $B \in \mathbb{R}^{r\times s}$ кронекерово произведение определяется как блочная матрица размера $pr \times qs$:

$$A \otimes B = \begin{bmatrix} A_{11}B & A_{12}B & \cdots & A_{1q}B \\ A_{21}B & A_{22}B & \cdots & A_{2q}B \\ \vdots & \vdots & \ddots & \vdots \\ A_{p1}B & A_{p2}B & \cdots & A_{pq}B \end{bmatrix}.$$

Ключевое тождество. Для произведения трёх матриц выполняется:

$$\mathrm{vec}(AXB) = (B^T \otimes A)\ \mathrm{vec}(X). \tag{10}$$

Сведение уравнения Ляпунова к СЛАУ. Применим тождество (10) к каждому слагае-

мому уравнения (9). Запишем левую часть, используя единичную матрицу I_n:

$$\text{vec}(A_{cl}^T X) = \text{vec}(A_{cl}^T \cdot X \cdot I_n) = (I_n \otimes A_{cl}^T)\ \text{vec}(X),$$
$$\text{vec}(X A_{cl}) = \text{vec}(I_n \cdot X \cdot A_{cl}) = (A_{cl}^T \otimes I_n)\ \text{vec}(X).$$

Складывая оба выражения, получаем СЛАУ размера $n^2 \times n^2$:

$$\underbrace{\left(I_n \otimes A_{cl}^T + A_{cl}^T \otimes I_n\right)}_{\mathcal{K}} \text{vec}(X) = \text{vec}(-F), \tag{11}$$

где матрица $\mathcal{K} \in \mathbb{R}^{n^2 \times n^2}$ называется *матрицей Кронекеровой суммы*.

Заполнение матрицы $\mathcal{K}$ в строчно-мажорной адресации. В реализации на языке C матрицы хранятся в строчном формате (row-major), поэтому элемент $X_{i,j}$ соответствует позиции $\text{vec}(X)[i \cdot n + j]$. Раскрывая определение Кронекеровой суммы поэлементно, для каждой строки $\text{row} = i \cdot n + j$ матрицы $\mathcal{K}$ ненулевые вклады дают два набора элементов:

$$\mathcal{K}_{(i\cdot n+j),\,(k\cdot n+j)} \mathrel{+}= (A_{cl})_{k,\,i}, \quad k = 0, \ldots, n-1 \quad \text{(вклад от слагаемого } I_n \otimes A_{cl}^T\text{)},$$
$$\mathcal{K}_{(i\cdot n+j),\,(i\cdot n+k)} \mathrel{+}= (A_{cl})_{k,\,j}, \quad k = 0, \ldots, n-1 \quad \text{(вклад от слагаемого } A_{cl}^T \otimes I_n\text{)}.$$

В каждой строке ненулевых элементов не более $2n$, что соответствует структуре уравнения Ляпунова.

Решение СЛАУ и извлечение результата. Система (11) решается методом Гаусса с частичным выбором ведущего элемента по столбцу (`gauss_solve`), после чего решение $\text{vec}(X)$ переформируется обратно в матрицу $X \in \mathbb{R}^{n \times n}$.

Вычислительная сложность. Построение матрицы $\mathcal{K}$ требует $O(n^3)$ операций. Решение СЛАУ методом Гаусса для матрицы размера $n^2 \times n^2$ требует $O(n^6)$ операций. Именно поэтому данный подход применяется только к *редуцированным* уравнениям малой размерности $l \ll N$: при $l = 5$ система имеет размер 25×25, при $l = 10$ — 100×100, что вычислительно приемлемо. Для исходной системы размера $N \times N$ прямое применение данного метода потребовало бы решения СЛАУ размера $N^2 \times N^2$, что и мотивирует использование проекционной редукции.

Пример для скалярного случая. При $n = 1$ уравнение Ляпунова $A_{cl}^T X + X A_{cl} = -F$ принимает вид $2A_{cl}X = -F$, откуда $X = -F/(2A_{cl})$. Матрица Кронекеровой суммы вырождается в скаляр $\mathcal{K} = 2A_{cl}$, что полностью согласуется с общей формулой.

Условие существования решения. Система (11) имеет единственное решение тогда и только тогда, когда матрица $\mathcal{K}$ невырождена, то есть когда у A_{cl} нет пар собственных значений (λ_i, λ_j) таких, что $\lambda_i + \lambda_j = 0$. На практике это условие обеспечивается устойчивостью A_{cl} (все $\text{Re}\,\lambda_i < 0$), которая гарантируется методом Ньютона–Клейнмана при выполнении условий стабилизируемости и обнаруживаемости исходной системы.

1.5 Метод Ньютона–Клейнмана и решатель уравнений Ляпунова

Внутри функции `solve_care_newton` выполняется итерационный поиск решения P_l:

1. Находится начальное приближение P_0 путём решения уравнения Ляпунова:

$$A_l^T P_0 + P_0 A_l = -C_l^T C_l$$

(функция `solve_lyapunov`) и последующей принудительной симметризации (`symmetrize`).

2. Запускается цикл итераций метода Ньютона–Клейнмана (до `MAX_NEWTON_ITERS` шагов). На каждом шаге k вычисляется новое приближение:

$$P_{k+1} = P_k + \Delta_k,$$

где поправка $\Delta_k \in \mathbb{R}^{l\times l}$ находится из уравнения Ляпунова:

$$A_{cl}^T \Delta_k + \Delta_k A_{cl} = -F(P_k),$$

причём:

$$A_{cl} = A_l - B_l B_l^T P_k \in \mathbb{R}^{l\times l} \quad \text{— замкнутая матрица системы,}$$
$$F(P_k) = A_l^T P_k + P_k A_l - P_k B_l B_l^T P_k + C_l^T C_l \in \mathbb{R}^{l\times l} \quad \text{— невязка на итерации } k.$$

Итерационный процесс прерывается, если $\|F(P_k)\|_F < \texttt{tol} = 10^{-10}$.

3. Для решения уравнения Ляпунова относительно несимметричной матрицы A_{cl} в функции `solve_lyapunov` строится СЛАУ Кронекеровой суммы размера $l^2 \times l^2$:

$$\mathcal{K} \cdot \mathrm{vec}(\Delta_k) = \mathrm{vec}(-F(P_k)),$$

где:

$$\mathcal{K} = I_l \otimes A_{cl}^T + A_{cl}^T \otimes I_l \in \mathbb{R}^{l^2\times l^2}.$$

С учётом строчно-мажорной адресации в C-коде коэффициенты матрицы $\mathcal{K}$ инкрементируются по правилу:

$$\mathcal{K}_{(il+j),\,(hl+j)} \leftarrow \mathcal{K}_{(il+j),\,(hl+j)} + (A_{cl})_{h,i},$$
$$\mathcal{K}_{(il+j),\,(il+h)} \leftarrow \mathcal{K}_{(il+j),\,(il+h)} + (A_{cl})_{h,j},$$

для всех $i, j, h = 0, \ldots, l-1$.

4. Полученная СЛАУ решается методом Гаусса с частичным выбором ведущего элемента по столбцу (`gauss_solve`).
5. Решение обновляется и симметризуется на каждом шаге.

1.6 Восстановление решения и оценка полной невязки

После сходимости редуцированного решения P_l программа оценивает его качество относительно исходной системы $N \times N$:

1. Вызывается функция `projected_full_residual`, которая восстанавливает полноразмерное приближение:
$$P^{(N)} = V_l P_l V_l^T \in \mathbb{R}^{N \times N}.$$
2. Вычисляется матрица невязки исходного уравнения Риккати:
$$R\big(P^{(N)}\big) = A^T P^{(N)} + P^{(N)} A - P^{(N)} B B^T P^{(N)} + C^T C \in \mathbb{R}^{N \times N}.$$
3. Рассчитывается норма Фробениуса невязки $\|R(P^{(N)})\|_F$.
4. На экран выводится строка отчёта: текущая размерность l, число итераций Ньютона и норма невязки.
5. Освобождается локальная динамическая память текущего шага l.

После прохода по всем $l = 1, \ldots, r_{use}$ программа освобождает память, занятую матрицами A, B, C и базисом V, и завершает работу.

1.7 Физический смысл матриц и примеры применения

1.7.1 Пример 1. Стабилизация обратного маятника

Задача состоит в удержании неустойчивого маятника в вертикальном положении за счёт движения тележки. Линеаризованная модель вблизи положения равновесия имеет вид (1)–(2) с вектором состояния $x = [s,\ \theta,\ \dot{s},\ \dot{\theta}]^T$, где s — смещение тележки, θ — угол отклонения маятника от вертикали.

Для типичных параметров ($M = 1{,}0$ кг, $m_p = 0{,}1$ кг, $L = 0{,}5$ м, $g = 9{,}81$ м/с^2, $d = 0{,}1$) матрицы системы принимают вид:

$$A \approx \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -0{,}981 & -0{,}1 & 0 \\ 0 & 21{,}582 & 0{,}2 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 1{,}0 \\ -2{,}0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

Положительный элемент $A_{42} = 21{,}582$ означает, что угловое ускорение маятника растёт при любом ненулевом угле — система неустойчива без управления. Матрица B показывает, что управляющая сила воздействует только на ускорения $\ddot{s}$ и $\ddot{\theta}$. Матрица C означает, что датчики измеряют только s (энкодер) и θ (гироскоп), но не скорости.

Решение уравнения Риккати $P \in \mathbb{R}^{4 \times 4}$ даёт матрицу регулятора $K = B^T P \in \mathbb{R}^{1 \times 4}$, при которой все собственные значения замкнутой матрицы $A - BK$ приобретают отрицательную вещественную часть, то есть система стабилизируется.

1.7.2 Пример 2. Управление температурой в трёхзонной промышленной печи

Задача — поддерживать заданный профиль температур в трёх зонах печи при минимальном расходе энергии. Вектор состояния $x = [T_1, T_2, T_3]^T$ содержит отклонения температур от заданного значения.

Для коэффициентов теплообмена $k_{12} = k_{21} = 0{,}5$, $k_{23} = k_{32} = 0{,}3$ и рассеяния $\alpha = 0{,}1$ матрицы принимают вид:

$$A = \begin{bmatrix} -0{,}6 & 0{,}5 & 0 \\ 0{,}5 & -0{,}9 & 0{,}3 \\ 0 & 0{,}3 & -0{,}4 \end{bmatrix}, \quad B = \begin{bmatrix} 1{,}0 & 0 \\ 0 & 0 \\ 0 & 1{,}0 \end{bmatrix}, \quad C = I_3.$$

Все диагональные элементы матрицы A по модулю превышают сумму остальных элементов в своей строке, поэтому все собственные значения A отрицательны — без нагревателей все зоны постепенно охлаждаются до температуры окружающей среды. Нулевая строка в B отражает факт, что 2-я зона не имеет собственного нагревателя и нагревается только через теплообмен с соседями. Единичная матрица $C = I_3$ означает, что термопары установлены во всех трёх зонах.

Решение уравнения Риккати даёт матрицу $K = B^T P \in \mathbb{R}^{2\times 3}$ — оптимальные коэффициенты обратной связи для двух нагревателей в зависимости от отклонений температур во всех трёх зонах.

1.8 Пример численного решения и проверка условий применимости

Рассмотрим простейшую одномерную неустойчивую систему ($N = 1$, $m = 1$, $p = 1$):

$$\dot{x}(t) = a\,x(t) + b\,u(t), \quad y(t) = c\,x(t),$$

с параметрами $A = [2]$, $B = [1]$, $C = [\sqrt{5}]$.

Проверка условий применимости.

— *Стабилизируемость* (6)*:* для единственного собственного значения $\lambda = 2 > 0$ матрица $[A - \lambda I \;\; B] = [2 - 2 \;\; 1] = [0 \;\; 1]$ имеет ранг $1 = N$. Условие выполнено: неустойчивая мода управляема.

— *Обнаруживаемость* (8)*:* матрица $\begin{bmatrix} A - \lambda I \\ C \end{bmatrix} = \begin{bmatrix} 0 \\ \sqrt{5} \end{bmatrix}$ имеет ранг $1 = N$. Условие выполнено: неустойчивая мода наблюдаема.

Таким образом, стабилизирующее решение $P > 0$ существует и единственно.

Аналитическое решение. Подставляя параметры в (4):

$$2P + P \cdot 2 - P \cdot 1 \cdot 1 \cdot P + 5 = 0 \implies P^2 - 4P - 5 = 0.$$

Корни: $P_{1,2} = \dfrac{4 \pm \sqrt{36}}{2}$, откуда $P_1 = 5$, $P_2 = -1$. Поскольку требуется $P > 0$, единственным

физически реализуемым решением является:

$$P = 5.$$

Синтез регулятора и проверка устойчивости. Матрица регулятора: $K = B^T P = 1 \cdot 5 = 5$, закон управления: $u(t) = -5\,x(t)$. Замкнутая матрица системы:

$$A_{cl} = A - BK = 2 - 1 \cdot 5 = -3.$$

Так как $A_{cl} = -3 < 0$, замкнутая система асимптотически устойчива:

$$x(t) = x(0)\,e^{-3t} \;\to\; 0 \quad \text{при} \quad t \to \infty.$$

Таким образом, оптимальное управление $u(t) = -5\,x(t)$ не только стабилизирует изначально неустойчивую систему ($A = 2 > 0$), но и минимизирует функционал качества (3).

2 ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ

Ниже представлена таблица с описанием использованных переменных.

Таблица 1 — Спецификация переменных программы

Название	**Тип**	**Границы типа**	**Назначение**
Параметры и структуры исходной динамической системы			
`N`	`int`	от 1 до $2^{31}-1$	Размерность вектора состояния исходной системы.
`m`	`int`	от 1 до $2^{31}-1$	Количество входов (размерность управления).
`p`	`int`	от 1 до $2^{31}-1$	Количество выходов (размерность измерения).
`r`	`int`	от 1 до $2^{31}-1$	Заданный максимальный ранг проекционного подпространства.
`steps`	`int`	от 1 до $2^{31}-1$	Число временных шагов для интегрирования траектории.
`Tfinal`	`double`	от $\pm 4.94 \times 10^{-324}$ до $\pm 1.79 \times 10^{308}$	Конечное время интегрирования траектории.
`A`	`double*`	Адрес памяти	Динамический массив элементов матрицы динамики исходной системы размера $N \times N$.
`B`	`double*`	Адрес памяти	Динамический массив элементов матрицы входов исходной системы размера $N \times m$.
`C`	`double*`	Адрес памяти	Динамический массив элементов матрицы выходов исходной системы размера $p \times N$.
Переменные этапа построения POD-базиса (функция `build_pod_basis`)			
`rmax`	`int`	от 1 до $2^{31}-1$	Ограничение размерности базиса: $\min(r, N)$.
`V`	`double*`	Адрес памяти	Динамический массив элементов матрицы полного ортонормированного базиса размера $N \times r_{max}$.
`r_eff`	`int`	от 0 до $2^{31}-1$	Фактическое число эффективных POD-мод с собственными значениями $\lambda_k > 10^{-12}$.

Название	**Тип**	**Границы типа**	**Назначение**
`Y`	`double*`	Адрес памяти	Траекторная матрица снимков сопряженного состояния размера $N \times S$, где $S = p \cdot (steps + 1)$.
`G`	`double*`	Адрес памяти	Симметричная корреляционная матрица Грамиана размера $S \times S$.
`U`	`double*`	Адрес памяти	Собственные векторы Грамиана размера $S \times S$ (направления POD-компонент).
`w`	`double*`	Адрес памяти	Массив собственных значений Грамиана размера S.
`x`	`double*`	Адрес памяти	Вектор текущего состояния сопряженной системы размера N.
`xnext`	`double*`	Адрес памяти	Вектор сопряженного состояния на следующем временном шаге.
`k1, k2, k3, k4`	`double*`	Адрес памяти	Вспомогательные векторы приращений интегрирования схемы Рунге-Кутты 4-го порядка размера N.
`xtmp`	`double*`	Адрес памяти	Временный вектор промежуточных вычислений шага Рунге-Кутты размера N.
Переменные этапа редукции размерности подпространства			
`r_use`	`int`	от 0 до $2^{31} - 1$	Фактически используемый ранг подпространства: $\min(r, r_{eff})$.
`l`	`int`	от 1 до $2^{31} - 1$	Управляющая переменная цикла — текущая размерность подпространства.
`Vl`	`double*`	Адрес памяти	Временная матрица проектора размера $N \times l$ (первые l столбцов матрицы V).
`Al`	`double*`	Адрес памяти	Динамический массив элементов спроецированной матрицы динамики порядка $l \times l$.
`Bl`	`double*`	Адрес памяти	Динамический массив элементов спроецированной матрицы входов порядка $l \times m$.

Название	**Тип**	**Границы типа**	**Назначение**
`Cl`	`double*`	Адрес памяти	Динамический массив элементов спроецированной матрицы выходов порядка $p \times l$.
`Pl`	`double*`	Адрес памяти	Решение редуцированного уравнения Риккати порядка $l \times l$.
`nit`	`int`	от 0 до $2^{31} - 1$	Число итераций метода Ньютона до достижения сходимости.
`res`	`double`	от 0.0 до 1.79×10^{308}	Норма Фробениуса полной невязки восстановленного решения Риккати.
Переменные решателя Ньютона-Клейнмана (функция `solve_care_newton`)			
`Q`	`double*`	Адрес памяти	Вспомогательная симметричная матрица $C^T C$ размера $n \times n$.
`S`	`double*`	Адрес памяти	Вспомогательная симметричная матрица BB^T размера $n \times n$.
`F`	`double*`	Адрес памяти	Невязка редуцированного уравнения Риккати порядка $n \times n$ на текущей итерации Ньютона.
`Acl`	`double*`	Адрес памяти	Замкнутая матрица редуцированной системы $A_l - B_l B_l^T P_k$ порядка $n \times n$.
`Delta`	`double*`	Адрес памяти	Поправка к решению Риккати на шаге Ньютона порядка $n \times n$ (решение уравнения Ляпунова).
Переменные Кронекерова решателя Ляпунова (функция `solve_lyapunov`)			
`K`	`double*`	Адрес памяти	Динамический массив векторизованной матрицы Кронекеровой суммы порядка $l^2 \times l^2$.
`rhs`	`double*`	Адрес памяти	Вектор правой части векторизованной СЛАУ Ляпунова размера l^2.
`sol`	`double*`	Адрес памяти	Вектор решения векторизованной СЛАУ Ляпунова размера l^2.

3 БЛОК-СХЕМА АЛГОРИТМА

Блок-схема алгоритма представлена в Приложении А.

4 ТЕСТИРОВАНИЕ ПРОГРАММЫ

```
ЛР5 ➤ ./lab5
Файл входных данных не указан. Введите данные в стандартный поток (stdin):
2 2 2 2 10 1.0
-1.0 0.0
0.0 -2.0
1.0 0.0
0.0 1.0
1.0 0.0
0.0 1.0
Параметры задачи: N=2 (полный размер), m=2 (входы), p=2 (выходы),
                  r=2 (целевой ранг), steps=10, T=1.0000

Эффективный ранг POD-базиса: 2
Используемый ранг:            2

l      Итер. Ньютона  ||R(P_l)||_F            Симм.  P>=0      Статус
-----  -------------  ----------------------  --------  --------  -------
1      3              1.000000e+00            ДА      ДА      OK
2      3              4.510836e-12            ДА      ДА      OK

========================================================================
Финальные результаты редуцированного уравнения Риккати (l = 2):
========================================================================
Редуцированная матрица P_l (2 x 2):
 4.142136e-01  0.000000e+00
 0.000000e+00  2.360680e-01

Восстановленная полноразмерная матрица P^(N) (2 x 2):
 4.142136e-01  0.000000e+00
 0.000000e+00  2.360680e-01

ЛР5 ➤ ./lab5 input.txt
Чтение данных из файла: input.txt
Параметры задачи: N=2 (полный размер), m=2 (входы), p=2 (выходы),
                  r=2 (целевой ранг), steps=10, T=1.0000

Эффективный ранг POD-базиса: 2
Используемый ранг:            2

l      Итер. Ньютона  ||R(P_l)||_F            Симм.  P>=0      Статус
-----  -------------  ----------------------  --------  --------  -------
1      3              1.000000e+00            ДА      ДА      OK
2      3              4.510836e-12            ДА      ДА      OK

========================================================================
Финальные результаты редуцированного уравнения Риккати (l = 2):
========================================================================
Редуцированная матрица P_l (2 x 2):
 4.142136e-01  0.000000e+00
 0.000000e+00  2.360680e-01

Восстановленная полноразмерная матрица P^(N) (2 x 2):
 4.142136e-01  0.000000e+00
 0.000000e+00  2.360680e-01

ЛР5 ➤ ./lab5 input2.txt
input2.txt: No such file or directory
ЛР5 ➤ ./lab5 input1.txt
Чтение данных из файла: input1.txt
Ошибка чтения параметров задачи.
```

```
Чтение данных из файла: TESTS/4σ4/formatted_input.txt
Параметры задачи: N=8 (полный размер), m=4 (входы), p=8 (выходы),
                  r=8 (целевой ранг), steps=100, T=1.0000

Эффективный ранг POD-базиса: 8
Используемый ранг:            8

l      Итер. Ньютона  ||R(P_l)||_F            Симм.  P>=0      Статус
-----  -------------  ---------------------  --------  --------  -------
1      5              2.047756e+01           ДА        ДА        OK
2      6              2.479233e+01           ДА        ДА        OK
3      50             1.131820e+02           ДА        ДА        НЕ СОШЁЛСЯ
4      37             1.213301e+03           ДА        ДА        OK
5      50             8.186400e+03           ДА        НЕТ       НЕ СОШЁЛСЯ
6      8              7.737507e+01           ДА        ДА        OK
7      7              3.150614e+00           ДА        ДА        OK
8      7              9.779942e-11           ДА        ДА        OK

======================================================================
Финальные результаты редуцированного уравнения Риккати (для l = 8):
======================================================================
Редуцированная матрица P_l (размер 8 x 8):
 2.117303e+01  5.200702e-01  9.897609e-01  2.467171e-01  1.113434e-01  5.281524e-01  1.026557e+00 -5.435105e-01
 5.200702e-01  1.849831e+01  6.114786e-01  9.823927e-02  5.764861e-02  2.138570e-01 -2.235581e-01 -7.596051e-01
 9.897609e-01  6.114786e-01  3.851330e+00  8.866216e-02  8.327398e-02 -2.114138e-01  1.872641e-01 -1.532506e-02
 2.467171e-01  9.823927e-02  8.866216e-02  2.753952e+00 -3.038423e-01 -7.502099e-03  4.309116e-02  3.795349e-03
 1.113434e-01  5.764861e-02  8.327398e-02 -3.038423e-01  7.633027e-01  4.452986e-03  4.514984e-03 -4.593543e-03
 5.281524e-01  2.138570e-01 -2.114138e-01 -7.502099e-03  4.452986e-03  5.642645e-01 -2.107158e-02 -2.000817e-02
 1.026557e+00 -2.235581e-01  1.872641e-01  4.309116e-02  4.514984e-03 -2.107158e-02  8.054389e-01 -4.604577e-02
-5.435105e-01 -7.596051e-01 -1.532506e-02  3.795349e-03 -4.593543e-03 -2.000817e-02 -4.604577e-02  1.980361e-01

Восстановленная полноразмерная матрица P (размер 8 x 8):
 9.531908e+00  6.903535e+00  3.428745e+00 -2.186263e+00  3.676288e+00  2.004960e+00 -4.722430e-03 -2.861113e+00
 6.903535e+00  8.165904e+00  3.944001e+00 -7.863638e-03  2.007343e+00  2.533087e+00 -1.105751e-01 -2.306172e+00
 3.428745e+00  3.944001e+00  6.610999e+00  4.903090e+00  3.522357e-03 -1.051568e-01  1.165161e+00 -7.736529e-01
-2.186263e+00 -7.863638e-03  4.903090e+00  1.561553e+01 -2.969399e+00 -2.481997e+00 -1.063147e+00  3.612944e+00
 3.676288e+00  2.007343e+00  3.522357e-03 -2.969399e+00  2.594641e+00  6.127516e-01 -1.157009e-01 -1.109759e+00
 2.004960e+00  2.533087e+00 -1.051568e-01 -2.481997e+00  6.127516e-01  2.175395e+00 -1.714676e-01 -9.638285e-01
-4.722430e-03 -1.105751e-01  1.165161e+00 -1.063147e+00 -1.157009e-01 -1.714676e-01  1.729492e+00 -5.444181e-01
-2.861113e+00 -2.306172e+00 -7.736529e-01  3.612944e+00 -1.109759e+00 -9.638285e-01 -5.444181e-01  2.183796e+00
.......
```

```
Чтение данных из файла: TESTS/6об6/formatted_input.txt
Параметры задачи: N=12 (полный размер), m=6 (входы), p=12 (выходы),
                  r=12 (целевой ранг), steps=50, T=1.0000

Эффективный ранг POD-базиса: 12
Используемый ранг:            12

l      Итер. Ньютона  ||R(P_l)||_F           Симм.  P>=0      Статус
-----  -------------  ---------------------  --------  --------  -------
1      5              1.604835e+01           ДА      ДА       OK
2      12             1.379210e+01           ДА      НЕТ    Р НЕКОРРЕКТНА
3      31             5.716793e+01           ДА      ДА       OK
4      27             1.344222e+01           ДА      НЕТ    Р НЕКОРРЕКТНА
5      8              1.361822e+01           ДА      НЕТ    Р НЕКОРРЕКТНА
6      8              1.369047e+01           ДА      НЕТ    Р НЕКОРРЕКТНА
7      8              1.382956e+01           ДА      НЕТ    Р НЕКОРРЕКТНА
8      8              1.403639e+01           ДА      НЕТ    Р НЕКОРРЕКТНА
9      8              1.533662e+01           ДА      ДА       OK
10     7              7.152429e+00           ДА      ДА       OK
11     7              3.654160e+00           ДА      ДА       OK
12     7              7.387495e-11           ДА      ДА       OK

========================================================================
Финальные результаты редуцированного уравнения Риккати (для l = 12):
========================================================================
Редуцированная матрица P_l (размер 12 x 12):
 3.299626e+01 -4.933659e-02  5.413440e-01  2.267157e+00  4.427202e-01  2.465190e-01  1.176884e-01
-4.933659e-02  2.919464e+01 -1.721615e+00 -1.723258e+00 -3.396644e-01 -1.618055e-01 -5.814530e-02
 5.413440e-01 -1.721615e+00  9.112314e+00  1.099248e+00  2.138712e-01  1.179109e-01  7.865782e-02
 2.267157e+00 -1.723258e+00  1.099248e+00  8.269186e+00  6.581593e-01  3.686287e-01  1.760386e-01
 4.427202e-01 -3.396644e-01  2.138712e-01  6.581593e-01  3.932559e+00  7.449809e-02  4.655389e-02
 2.465190e-01 -1.618055e-01  1.179109e-01  3.686287e-01  7.449809e-02  2.771687e+00 -2.968624e-01
 1.176884e-01 -5.814530e-02  7.865782e-02  1.760386e-01  4.655389e-02 -2.968624e-01  7.657619e-01
-8.538027e-02  3.339532e-02 -6.780210e-02 -1.669624e-01  3.484175e-01 -1.285286e-03 -3.038866e-03
-3.419520e-01  1.112314e+00 -1.906159e-01 -4.016997e-01 -7.922077e-02 -3.865288e-02 -1.052454e-02
 3.125458e-01 -5.327955e-01  2.040966e-01  6.799839e-01  1.186635e-01  5.795913e-02  1.137541e-02
 6.900786e-01 -6.900731e-03 -4.393355e-01  8.240444e-02  2.099716e-02  1.578403e-02  3.472004e-03
-5.256278e-01  6.381407e-01 -2.323646e-01 -2.523568e-02 -6.813979e-04  5.093815e-03 -9.467035e-05

Восстановленная полноразмерная матрица P (размер 12 x 12):
 1.178508e+01  8.821153e+00  4.956279e+00 -1.307461e+00  6.776495e+00 -2.250159e+00  4.085397e+00
 8.821153e+00  9.889976e+00  5.383094e+00  8.415276e-01  5.787417e+00  2.473162e-01  2.353983e+00
 4.956279e+00  5.383094e+00  7.830166e+00  5.577884e+00  3.455527e+00  4.035709e+00  3.121038e-01
-1.307461e+00  8.415276e-01  5.577884e+00  1.621540e+01 -8.087791e-01  9.919167e+00 -2.712086e+00
 6.776495e+00  5.787417e+00  3.455527e+00 -8.087791e-01  1.340788e+01 -1.339974e+00  2.147988e+00
-2.250159e+00  2.473162e-01  4.035709e+00  9.919167e+00 -1.339974e+00  1.737864e+01 -2.073793e+00
 4.085397e+00  2.353983e+00  3.121038e-01 -2.712086e+00  2.147988e+00 -2.073793e+00  2.650617e+00
 2.362067e+00  2.857127e+00  1.769261e-01 -2.308814e+00  1.502960e+00 -1.582633e+00  6.790168e-01
 3.245072e-01  1.798346e-01  1.384939e+00 -1.015663e+00  5.745574e-01 -8.006815e-01 -2.331246e-02
  ......
```

```
Чтение данных из файла: TESTS/10x10/formatted_input.txt
Параметры задачи: N=20 (полный размер), m=10 (входы), p=20 (выходы),
                  r=20 (целевой ранг), steps=40, T=1.0000

Эффективный ранг POD-базиса: 20
Используемый ранг:           20

l      Итер. Ньютона  ||R(P_l)||_F           Симм.  P>=0      Статус
-----  -------------  --------------------  --------  --------  -------
1      3              4.364599e+00          ДА      НЕТ     Р НЕКОРРЕКТНА
2      9              1.032497e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
3      10             1.131055e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
4      10             1.128469e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
5      10             1.113326e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
6      12             1.115394e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
7      12             1.117036e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
8      12             1.149437e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
9      50             6.921655e+03          ДА      НЕТ     НЕ СОШЁЛСЯ
10     37             1.256566e+04          ДА      НЕТ     Р НЕКОРРЕКТНА
11     48             9.080639e+03          ДА      НЕТ     Р НЕКОРРЕКТНА
12     37             8.691173e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
13     50             7.435290e+03          ДА      НЕТ     НЕ СОШЁЛСЯ
14     18             9.775742e+01          ДА      НЕТ     Р НЕКОРРЕКТНА
15     50             1.620754e+08          ДА      НЕТ     НЕ СОШЁЛСЯ
16     50             5.584823e+03          ДА      ДА       НЕ СОШЁЛСЯ
17     50             2.566788e+03          ДА      НЕТ     НЕ СОШЁЛСЯ
18     8              4.182202e+01          ДА      ДА       ОК
19     8              8.572128e+00          ДА      ДА       ОК
20     8              1.543195e-13          ДА      ДА       МАШ. ТОЧНОСТЬ

======================================================================
Финальные результаты редуцированного уравнения Риккати (для l = 20):
======================================================================
Редуцированная матрица P_l (размер 20 x 20):
 1.047923e+02  5.775073e-02  1.200348e-01  7.006544e-02 -1.263410e-01 -2.010118e-02 -1.503232e-02  2.851547e-02
 5.775073e-02  8.422812e+00  2.480098e-01  1.017397e-01 -1.806848e-01 -1.743470e-02 -1.258899e-02  4.489744e-02
 1.200348e-01  2.480098e-01  7.016233e+00  1.005220e-01 -1.882321e-01 -2.147255e-02 -1.556385e-02  5.168025e-02
 7.006544e-02  1.017397e-01  1.005220e-01  4.149632e+00 -3.647730e-02 -9.689407e-03 -7.204240e-03  1.182190e-02
-1.263410e-01 -1.806848e-01 -1.882321e-01 -3.647730e-02  3.462175e+00  1.964202e-02  1.469313e-02 -1.510070e-02
-2.010118e-02 -1.743470e-02 -2.147255e-02 -9.689407e-03  1.964202e-02  2.645028e+00  3.585662e-04 -6.382407e-03
-1.503232e-02 -1.258899e-02 -1.556385e-02 -7.204240e-03  1.469313e-02  3.585662e-04  2.588886e+00 -4.906738e-03
 2.851547e-02  4.489744e-02  5.168025e-02  1.182190e-02 -1.510070e-02 -6.382407e-03 -4.906738e-03  2.074601e+00
 7.244787e-03 -4.532456e-02 -5.512696e-02 -4.705274e-03 -1.127245e-02  1.473713e-02  1.178011e-02  2.059162e-02
 6.473232e-02  8.733259e-02  1.032499e-01  2.851487e-02 -4.093546e-02 -1.217895e-02 -9.354311e-03 -1.772316e-03
 3.244653e-03 -7.005413e-02 -8.538169e-02 -1.159656e-03 -4.634243e-02  1.606309e-02  1.303846e-02  1.653099e-01
-1.220267e-02 -5.004093e-02 -5.879679e-02 -6.028062e-04 -3.432161e-02  6.764554e-03  5.509569e-03 -5.690702e-01
-1.737621e-02 -1.849414e-02 -2.245744e-02 -1.224097e-02  3.493301e-02  3.602692e-02  3.160624e-01  4.175873e-03
-1.801421e-02 -1.998760e-02 -2.424703e-02 -1.294693e-02  3.772876e-02  3.159442e-01 -3.307665e-02  3.035214e-03
-1.987332e-03  4.997075e-02  4.810282e-02 -1.140752e-01 -6.167360e-01 -1.278872e-02 -9.458257e-03  2.120009e-02
  ......

Восстановленная полноразмерная матрица P (размер 20 x 20):
 8.262090e+00  4.869232e+00  9.721011e+00  6.182309e+00  6.603982e+00  7.751769e+00  6.212660e+00  8.503781e+00
 4.869232e+00  1.136182e+01  1.146604e+01  8.058793e+00  8.496972e+00  8.016135e+00  1.017998e+01  8.728221e+00
 9.721011e+00  1.146604e+01  2.297384e+01  1.503904e+01  1.483988e+01  1.193915e+01  1.475778e+01  1.650226e+01
 6.182309e+00  8.058793e+00  1.503904e+01  1.339005e+01  1.037320e+01  7.732090e+00  1.089017e+01  1.093131e+01
 6.603982e+00  8.496972e+00  1.483988e+01  1.037320e+01  1.268357e+01  8.818034e+00  1.133534e+01  1.114725e+01
 7.751769e+00  8.016135e+00  1.193915e+01  7.732090e+00  8.818034e+00  1.254384e+01  9.787735e+00  9.967734e+00
 6.212660e+00  1.017998e+01  1.475778e+01  1.089017e+01  1.133534e+01  9.787735e+00  1.609591e+01  1.102267e+01
 8.503781e+00  8.728221e+00  1.650226e+01  1.093131e+01  1.114725e+01  9.967734e+00  1.102267e+01  1.618819e+01
 5.558908e+00  8.847585e+00  1.332764e+01  9.226197e+00  1.029915e+01  8.491558e+00  1.251975e+01  1.096229e+01
 7.042602e+00  7.013225e+00  1.203674e+01  8.057304e+00  8.783913e+00  9.270736e+00  9.411021e+00  9.519350e+00
 5.879625e-01  1.473064e-01 -9.012751e-02  4.498355e-02  2.557522e-02 -3.539196e-01 -1.091933e-03 -1.478792e-01
 1.069965e-01  1.639252e-01 -6.429481e-02  5.658228e-03  5.439886e-03 -1.383514e-01 -4.872514e-02 -4.405599e-02
-1.268408e-01 -6.155941e-02  4.772655e-01 -1.749138e-01 -2.986006e-01  5.522803e-02 -3.858572e-02 -2.012222e-01
 2.067178e-02  2.899435e-03 -1.881330e-01  2.240852e-01  1.903206e-04 -3.475552e-03 -6.625209e-02 -1.072791e-02
 2.527071e-02  8.730636e-03 -2.994737e-01  4.076152e-03  4.539858e-01 -4.394115e-02  9.591324e-03  1.004328e-01
-3.530881e-01 -7.889000e-02  9.875384e-02  2.126697e-02 -5.134578e-02  2.710872e-01 -4.744461e-02  1.016641e-01
 1.853352e-02 -4.665481e-02 -4.656304e-02 -7.280753e-02  5.870520e-03 -1.024726e-02  2.417281e-01  7.983009e-02
-2.198699e-01 -4.332607e-02 -1.921105e-01  1.148022e-02  1.047247e-01 -2.677754e-03  8.723992e-02  2.815386e-01
 1.020560e-01 -2.951571e-03  6.244836e-02  2.130974e-02 -1.406002e-01 -6.663931e-02 -1.660081e-01 -1.884617e-01
 9.138892e-02  2.263998e-02  6.637477e-02  2.834824e-02  7.949266e-03  1.773600e-01  4.901919e-03  3.713522e-02
  ......
```

```
Чтение данных из файла: TESTS/40x40/formatted_input.txt
Параметры задачи: N=80 (полный размер), m=40 (входы), p=40 (выходы),
                  r=10 (целевой ранг), steps=10, T=1.0000

Эффективный ранг POD-базиса: 10
Используемый ранг:            10

l      Итер. Ньютона  ||R(P_l)||_F            Симм.  P>=0      Статус
-----  -------------  ----------------------  --------  --------  -------
1      7              7.621315e+00            ДА      ДА      OK
2      7              8.726747e+00            ДА      ДА      OK
3      7              9.704947e+00            ДА      ДА      OK
4      7              1.059108e+01            ДА      ДА      OK
5      7              1.140279e+01            ДА      ДА      OK
6      7              1.215764e+01            ДА      ДА      OK
7      7              1.286561e+01            ДА      ДА      OK
8      7              1.353041e+01            ДА      ДА      OK
9      7              1.415982e+01            ДА      ДА      OK
10     7              1.475795e+01            ДА      ДА      OK

========================================================================
Финальные результаты редуцированного уравнения Риккати (для l = 10):
========================================================================
Редуцированная матрица P_l (размер 10 x 10):
 4.505957e+00  1.721985e-06 -3.188885e-08 -5.230863e-06  1.522836e-06  3.978498e-07  1.253341e-06  8.962104e-07
 1.721985e-06  4.503873e+00 -8.157919e-07 -5.011611e-06  1.958639e-06  4.367205e-06  1.524792e-05  6.448914e-08
-3.188885e-08 -8.157919e-07  4.497048e+00 -4.010924e-07 -5.019817e-07 -2.791178e-07 -5.305369e-06 -3.980025e-07
-5.230863e-06 -5.011611e-06 -4.010924e-07  4.489709e+00  7.360925e-07 -3.681202e-06  8.069771e-08  1.239082e-06
 1.522836e-06  1.958639e-06 -5.019817e-07  7.360925e-07  4.468117e+00 -2.262244e-06 -1.327755e-05 -3.984377e-06
 3.978498e-07  4.367205e-06 -2.791178e-07 -3.681202e-06 -2.262244e-06  4.457001e+00  2.046151e-07  1.016686e-06
 1.253341e-06  1.524792e-05 -5.305369e-06  8.069771e-08 -1.327755e-05  2.046151e-07  4.445785e+00 -7.218628e-08
 8.962104e-07  6.448914e-08 -3.980025e-07  1.239082e-06 -3.984377e-06  1.016686e-06 -7.218628e-08  4.418390e+00
-7.607303e-07  1.133218e-06 -1.273329e-07 -1.900219e-06  3.559059e-07 -1.097971e-06  1.578003e-06  1.421844e-05
-9.243702e-07 -1.112519e-06  7.630232e-07 -1.031555e-06 -9.567052e-07 -1.749564e-07 -1.597761e-06  9.271508e-06

Восстановленная полноразмерная матрица P (размер 80 x 80):
 7.749851e-01 -1.357749e-01 -2.091630e-01  2.648091e-01 -2.681383e-01  9.669367e-02  8.937093e-02  9.475539e-03
-1.357749e-01  4.983135e-01  2.035644e-01  1.266742e-01 -4.334610e-01 -2.296797e-01  1.155934e-01 -1.644449e-01
-2.091630e-01  2.035644e-01  8.514866e-01  2.657944e-01 -1.063062e-01 -1.498702e-01 -1.745569e-01  1.488547e-02
 2.648091e-01  1.266742e-01  2.657944e-01  3.533047e-01 -2.659061e-01 -7.463150e-02 -8.643205e-02 -4.440339e-02
-2.681383e-01 -4.334610e-01 -1.063062e-01 -2.659061e-01  1.115601e+00 -2.707678e-01 -1.399265e-01 -8.041251e-02
 9.669367e-02 -2.296797e-01 -1.498702e-01 -7.463150e-02 -2.707678e-01  8.812932e-01 -3.041383e-01  4.145040e-01
 8.937093e-02  1.155934e-01 -1.745569e-01 -8.643205e-02 -1.399265e-01 -3.041383e-01  6.622533e-01 -2.491971e-02
 9.475539e-03 -1.644449e-01  1.488547e-02 -4.440339e-02 -8.041251e-02  4.145040e-01 -2.491971e-02  7.845151e-01
-3.717212e-01  3.557174e-01  6.163538e-01  2.311001e-01 -2.107161e-02 -2.121416e-01 -2.489671e-01 -1.470728e-01
-4.034746e-01 -2.752787e-01  3.509407e-03 -3.706526e-01  2.630683e-01  4.416024e-02  2.465089e-02  1.164912e-01
 2.860120e-02  3.158403e-02 -1.232452e-01 -6.808918e-02  2.724174e-01 -1.320477e-01  2.225986e-01  1.760166e-01
-1.455865e-01 -2.330235e-01  1.875055e-01  4.995765e-03  3.890209e-01 -1.740564e-02 -8.386254e-02  2.860272e-01
 2.671274e-01  1.137505e-01 -2.743295e-01 -1.302851e-02 -5.251587e-01  2.037277e-02  2.933381e-01 -1.154081e-01
  .....
```

```
Чтение данных из файла: TESTS/60x60/formatted_input.txt
Параметры задачи: N=120 (полный размер), m=60 (входы), p=120 (выходы),
                  r=10 (целевой ранг), steps=10, T=1.0000

Эффективный ранг POD-базиса: 10
Используемый ранг:            10

l      Итер. Ньютона  ||R(P_l)||_F            Симм.  P>=0      Статус
-----  -------------  ----------------------  --------  --------  -------
1      7              2.507338e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
2      7              2.510935e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
3      7              2.511433e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
4      8              2.515637e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
5      8              2.522731e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
6      8              2.524983e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
7      8              2.529029e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
8      8              2.529261e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
9      8              2.541437e+01            ДА      НЕТ    Р НЕКОРРЕКТНА
10     8              2.543872e+01            ДА      НЕТ    Р НЕКОРРЕКТНА

========================================================================
Финальные результаты редуцированного уравнения Риккати (для l = 10):
========================================================================
Редуцированная матрица P_l (размер 10 x 10):
-1.624481e+01 -1.162961e-01  3.200342e-02 -1.251599e-01 -1.640150e-01 -8.915510e-02  1.232556e-01 -5.767345e-03
-1.162961e-01  2.152041e+00 -1.187064e-03 -6.243882e-04 -4.507314e-04  1.456328e-04  3.537924e-04 -2.024229e-03
 3.200342e-02 -1.187064e-03  2.150525e+00  1.149092e-03  3.664806e-03  2.603891e-03  2.393159e-03  9.219134e-04
-1.251599e-01 -6.243882e-04  1.149092e-03  2.149809e+00  2.958676e-04  1.207865e-03 -2.281339e-03  5.080209e-03
-1.640150e-01 -4.507314e-04  3.664806e-03  2.958676e-04  2.149308e+00 -2.923188e-04  1.151115e-03  1.041812e-03
-8.915510e-02  1.456328e-04  2.603891e-03  1.207865e-03 -2.923188e-04  2.148910e+00 -2.482237e-04 -2.285487e-03
 1.232556e-01  3.537924e-04  2.393159e-03 -2.281339e-03  1.151115e-03 -2.482237e-04  2.132293e+00  1.767429e-03
-5.767345e-03 -2.024229e-03  9.219134e-04  5.080209e-03  1.041812e-03 -2.285487e-03  1.767429e-03  2.127175e+00
 2.156058e-01  6.251471e-04 -9.123637e-04  1.593364e-05  2.336465e-03  1.349771e-03  5.112843e-03 -3.859223e-03
-9.397848e-02  2.544018e-05 -3.012665e-04  1.281495e-03  3.391317e-04 -3.412877e-03  2.675877e-04 -5.365008e-03

Восстановленная полноразмерная матрица P (размер 120 x 120):
-2.799287e-01 -3.245633e-01 -2.834688e-01 -3.033495e-01 -3.516766e-01 -3.732486e-01 -2.736144e-01 -3.143768e-01
-3.245633e-01 -2.480808e-01 -2.712634e-01 -2.859019e-01 -2.617370e-01 -2.938161e-01 -2.781250e-01 -2.813808e-01
-2.834688e-01 -2.712634e-01 -1.385061e-01 -2.407945e-01 -2.015479e-01 -2.991498e-01 -2.919822e-01 -2.575196e-01
-3.033495e-01 -2.859019e-01 -2.407945e-01 -1.303507e-01 -2.321520e-01 -2.660725e-01 -3.521854e-01 -2.432790e-01
-3.516766e-01 -2.617370e-01 -2.015479e-01 -2.321520e-01 -1.348279e-01 -3.150564e-01 -3.324560e-01 -2.663462e-01
-3.732486e-01 -2.938161e-01 -2.991498e-01 -2.660725e-01 -3.150564e-01 -2.515565e-01 -3.333177e-01 -2.684384e-01
-2.736144e-01 -2.781250e-01 -2.919822e-01 -3.521854e-01 -3.324560e-01 -3.333177e-01 -2.883099e-02 -2.394794e-01
-3.143768e-01 -2.813808e-01 -2.575196e-01 -2.432790e-01 -2.663462e-01 -2.684384e-01 -2.394794e-01 -9.448420e-02
-2.315414e-01 -2.729481e-01 -2.931527e-01 -3.032701e-01 -3.833054e-01 -2.856248e-01 -2.009659e-01 -3.039605e-01
-2.558304e-01 -2.281634e-01 -2.216362e-01 -2.399834e-01 -2.749966e-01 -2.590215e-01 -2.077803e-01 -2.612561e-01
-3.132246e-01 -2.130008e-01 -2.966852e-01 -2.678996e-01 -2.715633e-01 -2.174546e-01 -2.430428e-01 -1.507368e-01
-3.215031e-01 -3.036028e-01 -2.965681e-01 -2.856431e-01 -3.578179e-01 -3.092850e-01 -2.281007e-01 -2.347282e-01
-3.007075e-01 -3.225352e-01 -2.298794e-01 -3.328262e-01 -3.566869e-01 -3.077227e-01 -2.290258e-01 -2.461704e-01
  ......
```

```
Чтение данных из файла: TESTS/100X100/formatted_input.txt
Параметры задачи: N=200 (полный размер), m=100 (входы), p=200 (выходы),
                  r=15 (целевой ранг), steps=15, T=1.0000

Эффективный ранг POD-базиса: 15
Используемый ранг:           15

l      Итер. Ньютона  ||R(P_l)||_F           Симм.  P>=0      Статус
-----  -------------  ---------------------  --------  --------  -------
1      7              1.410975e+01           ДА      ДА      OK
2      7              1.411277e+01           ДА      ДА      OK
3      7              1.411578e+01           ДА      ДА      OK
4      7              1.411880e+01           ДА      ДА      OK
5      8              1.412181e+01           ДА      ДА      OK
6      8              1.412482e+01           ДА      ДА      OK
7      8              1.412784e+01           ДА      ДА      OK
8      8              1.413085e+01           ДА      ДА      OK
9      8              1.413386e+01           ДА      ДА      OK
10     8              1.413687e+01           ДА      ДА      OK
11     8              1.413988e+01           ДА      ДА      OK
12     8              1.414289e+01           ДА      ДА      OK
13     8              1.414590e+01           ДА      ДА      OK
14     8              1.414891e+01           ДА      ДА      OK
15     8              1.415191e+01           ДА      ДА      OK

========================================================================
Финальные результаты редуцированного уравнения Риккати (для l = 15):
========================================================================
Редуцированная матрица P_l (размер 15 x 15):
 2.150647e+00 -5.371869e-164 -4.696173e-164 -5.111168e-164 -4.389361e-164 -4.497544e-164 -4.816770e-164
-5.371869e-164  2.150647e+00 -5.141317e-164 -5.102300e-164 -4.336157e-164 -4.889483e-164 -4.825637e-164
-4.696173e-164 -5.141317e-164  2.150647e+00 -4.623461e-164 -3.908749e-164 -4.446113e-164 -4.224428e-164
-5.111168e-164 -5.102300e-164 -4.623461e-164  2.150647e+00 -4.123340e-164 -4.105605e-164 -4.345024e-164
-4.389361e-164 -4.336157e-164 -3.908749e-164 -4.123340e-164  2.150647e+00 -4.015157e-164 -4.054174e-164
-4.497544e-164 -4.889483e-164 -4.446113e-164 -4.105605e-164 -4.015157e-164  2.150647e+00 -4.295367e-164
-4.816770e-164 -4.825637e-164 -4.224428e-164 -4.345024e-164 -4.054174e-164 -4.295367e-164  2.150647e+00
-5.004759e-164 -5.276101e-164 -4.481582e-164 -4.634102e-164 -4.439019e-164 -4.520599e-164 -4.611046e-164
-4.345024e-164 -4.355665e-164 -3.947765e-164 -3.985008e-164 -3.598390e-164 -3.687064e-164 -3.672876e-164
-4.509958e-164 -4.653610e-164 -4.440792e-164 -4.568483e-164 -3.798793e-164 -3.765097e-164 -3.912295e-164
-4.650063e-164 -5.054416e-164 -4.272312e-164 -4.412417e-164 -3.995649e-164 -4.130433e-164 -4.318422e-164
-5.081018e-164 -5.045549e-164 -4.550748e-164 -4.749378e-164 -3.999196e-164 -4.501091e-164 -4.658930e-164
-4.889483e-164 -5.105847e-164 -4.446113e-164 -4.463848e-164 -4.194279e-164 -4.545428e-164 -4.412417e-164
-4.343251e-164 -4.644742e-164 -4.233295e-164 -4.584444e-164 -3.717213e-164 -4.258124e-164 -4.157036e-164
-5.494239e-164 -5.378963e-164 -5.061510e-164 -5.308024e-164 -4.474488e-164 -4.933820e-164 -4.811450e-164

Восстановленная полноразмерная матрица P (размер 200 x 200):
 7.365833e-01 -3.229546e-164 -2.823320e-164 -3.072813e-164 -2.638866e-164 -2.703905e-164 -2.895823e-164
-3.229546e-164  7.365833e-01 -3.090939e-164 -3.067482e-164 -2.606880e-164 -2.939537e-164 -2.901154e-164
-2.823320e-164 -3.090939e-164  7.365833e-01 -2.779606e-164 -2.349924e-164 -2.672985e-164 -2.539709e-164
  ......
```

5 ИСХОДНЫЙ КОД ПРОГРАММЫ

Программа написана на языке C, для компиляции использовался компилятор gcc.

Исходный код Makefile:

```
CC = gcc
CFLAGS = -O3 -Wall -Wextra -Werror -fanalyzer
LIBS = -lm
TARGET = lab5

all: $(TARGET)

$(TARGET): lab5.c
    $(CC) $(CFLAGS) $(TARGET).c -o $(TARGET) $(LIBS)

clean:
    rm -f $(TARGET)
```

Исходный код lab5ċ:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

/* Макрос IDX переводит двумерный индекс (i, j) матрицы в одномерный индекс
 * массива при строчно-мажорном (row-major) хранении, как принято в языке C.
 * Параметр ld (leading dimension) — количество столбцов в строке матрицы.
 * Пример: для матрицы 3x4 элемент [1][2] => IDX(1,2,4) = 1*4+2 = 6. */
#define IDX(i,j,ld) ((i)*(ld) + (j))

/* Порог отсечения собственных значений матрицы Грамиана при построении POD-базиса.
 * Собственные значения λ_k <= EPS_EIG считаются численным шумом
 * и соответствующие им POD-моды в базис не включаются. */
#define EPS_EIG 1e-12

/* Максимальное число итераций метода вращений Якоби.
 * Используется при нахождении собственных значений и векторов
 * симметричной матрицы Грамиана G. При достижении предела
 * алгоритм завершается с текущей точностью. */
#define MAX_JACOBI_ITERS 50000

/* Максимальное число итераций метода Ньютона-Клейнмана.
 * Если за MAX_NEWTON_ITERS шагов алгоритм не достиг заданного
 * допуска tol, итерации прерываются с предупреждением. */
#define MAX_NEWTON_ITERS 50

/* Флаг управления выводом предупреждений в stderr.
 *   1 — выводить предупреждения (режим по умолчанию);
 *   0 — подавить все предупреждения (тихий режим, флаг -s/--silent).
 * Предупреждения информируют о росте невязки, несходимости метода
 * Ньютона, несимметричности или неположительности результата P. */
static int show_warnings = 0;



/* xcalloc — безопасное выделение обнулённого блока памяти размером nbytes байт.
 * В отличие от стандартного calloc, при нехватке памяти выводит сообщение
 * об ошибке и немедленно завершает программу, исключая разыменование NULL. */
static void *xcalloc(size_t nbytes) {
    void *p = calloc(1, nbytes);
    if (!p) {
        fprintf(stderr, "Ошибка: Недостаточно оперативной памяти.\n");
```

```
        exit(EXIT_FAILURE);
    }
    return p;
}

/* fro_norm — норма Фробениуса матрицы (или вектора), переданной как
 * одномерный массив A длины n:
 *   ||A||_F = sqrt( sum_{i=0}^{n-1} A[i]^2 ).
 * Для матрицы M (k x l) вызывается как fro_norm(M, k*l). */
static double fro_norm(const double *A, int n) {
    double s = 0.0;
    for (int i = 0; i < n; ++i) s += A[i] * A[i];
    return sqrt(s);
}

/* symmetrize — приведение квадратной матрицы A (n x n) к симметричному виду
 * по формуле A := (A + A^T) / 2.
 * Применяется после каждого шага метода Ньютона-Клейнмана для компенсации
 * численной несимметрии, накапливаемой при решении уравнений Ляпунова. */
static void symmetrize(double *A, int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            double v = 0.5 * (A[IDX(i,j,n)] + A[IDX(j,i,n)]);
            A[IDX(i,j,n)] = v;
            A[IDX(j,i,n)] = v;
        }
    }
}

/* matmul — умножение матриц общего вида: C = A * B.
 * Размерности: A (n x m), B (m x k), результат C (n x k).
 * Внутри используется пропуск нулевых элементов A для ускорения
 * операции на блочно-разреженных матрицах (например, матрица A
 * системы 2-го порядка содержит блок нулей). */
static void matmul(const double *A, const double *B, double *C,
                   int n, int m, int k) {
    for (int i = 0; i < n * k; ++i) C[i] = 0.0;
    for (int i = 0; i < n; ++i) {
        for (int t = 0; t < m; ++t) {
            double a = A[IDX(i,t,m)];
            if (a == 0.0) continue;
            for (int j = 0; j < k; ++j)
                C[IDX(i,j,k)] += a * B[IDX(t,j,k)];
        }
    }
}

/* at_mul_b — умножение транспонированной матрицы A^T на матрицу B: C = A^T * B.
 * Транспонирование выполняется неявно (без копирования), что экономит память.
 * Размерности: A хранится как (n x m), используется как (m x n);
 * B (n x k), результат C (m x k).
 * Применяется для вычисления:
 *   Al = Vl^T * AV (проецирование матрицы динамики),
 *   Bl = Vl^T * B  (проецирование матрицы входов),
 *   AP = A^T * P   (слагаемое невязки CARE). */
static void at_mul_b(const double *A, const double *B, double *C,
                     int n, int m, int k) {
    for (int i = 0; i < m * k; ++i) C[i] = 0.0;
    for (int i = 0; i < n; ++i) {
        for (int t = 0; t < m; ++t) {
            double a = A[IDX(i,t,m)];
            if (a == 0.0) continue;
            for (int j = 0; j < k; ++j)
                C[IDX(t,j,k)] += a * B[IDX(i,j,k)];
        }
    }
}

/* matmul_right_transpose — умножение матрицы A на транспонированную B^T: C = A * B^T.
```

```
 * Транспонирование B выполняется неявно.
 * Размерности: A (n x m), B хранится как (k x m), используется как (m x k);
 * результат C (n x k).
 * Применяется для вычисления:
 *   S = B * B^T (матрица управляющей ковариации),
 *   P^(N) = tmp * Vl^T (восстановление полноразмерного решения). */
static void matmul_right_transpose(const double *A, const double *B,
                                   double *C, int n, int m, int k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < k; ++j) {
            double s = 0.0;
            for (int t = 0; t < m; ++t)
                s += A[IDX(i,t,m)] * B[IDX(j,t,m)];
            C[IDX(i,j,k)] = s;
        }
    }
}

/* compute_bbt — вычисление матрицы S = B * B^T для матрицы входов B (n x m).
 * Результат S (n x n) — симметричная положительно полуопределённая матрица.
 * Соответствует весовой матрице управления в уравнении Риккати:
 *   член -P * B * B^T * P отвечает за «стоимость» управляющего воздействия. */
static void compute_bbt(const double *B, int n, int m, double *S) {
    matmul_right_transpose(B, B, S, n, m, n);
}

/* compute_ctc — вычисление матрицы Q = C^T * C для матрицы выходов C (p x n).
 * Результат Q (n x n) — симметричная положительно полуопределённая матрица.
 * Соответствует штрафной матрице состояния в функционале LQR:
 *   J = integral( x^T Q x + u^T u ) dt,  Q = C^T C. */
static void compute_ctc(const double *C, int p, int n, double *Q) {
    at_mul_b(C, C, Q, p, n, n);
}



/* apply_AT — умножение вектора x на транспонированную матрицу системы: y = A^T * x.
 * Используется как правая часть сопряжённой системы dx/dt = A^T * x
 * при построении траекторий для POD-базиса. */
static void apply_AT(const double *A, const double *x,
                     double *y, int n) {
    for (int i = 0; i < n; ++i) {
        double s = 0.0;
        for (int j = 0; j < n; ++j)
            s += A[IDX(j,i,n)] * x[j];
        y[i] = s;
    }
}

/* rk4_step_AT — один шаг численного интегрирования сопряжённой линейной системы
 *   dx/dt = A^T * x
 * классическим методом Рунге-Кутты 4-го порядка (RK4).
 *
 * Формула одного шага:
 *   k1 = A^T * x(t)
 *   k2 = A^T * (x + 0.5*dt*k1)
 *   k3 = A^T * (x + 0.5*dt*k2)
 *   k4 = A^T * (x + dt*k3)
 *   x(t+dt) = x(t) + (dt/6) * (k1 + 2*k2 + 2*k3 + k4)
 *
 * Рабочие буферы k1..k4 и xtmp передаются снаружи, чтобы не
 * выделять память заново на каждом из тысяч временных шагов. */
static void rk4_step_AT(const double *A, const double *x,
                        double dt, double *xnext, int n,
                        double *k1, double *k2,
                        double *k3, double *k4, double *xtmp) {
    /* Первый наклон: k1 = A^T * x(t) */
    apply_AT(A, x, k1, n);
    for (int i = 0; i < n; ++i) xtmp[i] = x[i] + 0.5 * dt * k1[i];
```

```
    /* Второй наклон: k2 = A^T * (x + dt/2 * k1) */
    apply_AT(A, xtmp, k2, n);
    for (int i = 0; i < n; ++i) xtmp[i] = x[i] + 0.5 * dt * k2[i];

    /* Третий наклон: k3 = A^T * (x + dt/2 * k2) */
    apply_AT(A, xtmp, k3, n);
    for (int i = 0; i < n; ++i) xtmp[i] = x[i] + dt * k3[i];

    /* Четвёртый наклон: k4 = A^T * (x + dt * k3) */
    apply_AT(A, xtmp, k4, n);

    /* Итоговое взвешенное обновление состояния */
    for (int i = 0; i < n; ++i)
        xnext[i] = x[i] + (dt / 6.0) *
                   (k1[i] + 2.0*k2[i] + 2.0*k3[i] + k4[i]);
}



/* jacobi_sym_eigen — нахождение полного спектра симметричной матрицы A (n x n)
 * итерационным методом вращений Якоби.
 *
 * На каждой итерации:
 *   1. Находится максимальный по модулю внедиагональный элемент A[p,q].
 *   2. Вычисляются параметры плоского вращения Якоби (c, s):
 *        tau = (A[q,q] - A[p,p]) / (2 * A[p,q])
 *        t   = sign(tau) / (|tau| + sqrt(1 + tau^2))
 *        c   = 1/sqrt(1+t^2),   s = t*c
 *   3. Применяется ортогональное преобразование подобия, обнуляющее A[p,q].
 *   4. Накапливается произведение поворотов в матрице собственных векторов V.
 *
 * Алгоритм сходится, когда max |A[i,j]| < 1e-14 для i != j.
 *
 * Выход:
 *   w[i] — собственные значения (считываются с диагонали после сходимости),
 *   V    — матрица собственных векторов (k-й столбец — вектор для λ_k). */
static void jacobi_sym_eigen(double *A, double *V, double *w, int n) {
    /* Инициализация V = I (единичная матрица — нейтральный элемент для поворотов) */
    for (int i = 0; i < n * n; ++i) V[i] = 0.0;
    for (int i = 0; i < n; ++i) V[IDX(i,i,n)] = 1.0;

    for (int iter = 0; iter < MAX_JACOBI_ITERS; ++iter) {
        /* Поиск максимального внедиагонального элемента */
        int p = 0, q = 1;
        double max_off = fabs(A[IDX(0,1,n)]);
        for (int i = 0; i < n - 1; ++i) {
            for (int j = i + 1; j < n; ++j) {
                double val = fabs(A[IDX(i,j,n)]);
                if (val > max_off) {
                    max_off = val;
                    p = i; q = j;
                }
            }
        }
        /* Критерий сходимости: все внедиагональные элементы пренебрежимо малы */
        if (max_off < 1e-14) break;

        double app = A[IDX(p,p,n)];
        double aqq = A[IDX(q,q,n)];
        double apq = A[IDX(p,q,n)];
        /* Пропуск вырожденного случая (теоретически невозможен при max_off > 0) */
        if (fabs(apq) < 1e-30) continue;

        /* Параметры вращения Якоби в плоскости (p, q) */
        double tau = (aqq - app) / (2.0 * apq);
        double t   = (tau >= 0.0 ? 1.0 : -1.0) /
                     (fabs(tau) + sqrt(1.0 + tau * tau));
        double c   = 1.0 / sqrt(1.0 + t * t);  /* косинус угла поворота */
```

```
        double s   = t * c;                    /* синус угла поворота  */

        /* Обновление строк и столбцов k != p, q в матрице A */
        for (int k = 0; k < n; ++k) {
            if (k == p || k == q) continue;
            double aik    = A[IDX(k,p,n)];
            double akq    = A[IDX(k,q,n)];
            double new_kp = c * aik - s * akq;
            double new_kq = s * aik + c * akq;
            A[IDX(k,p,n)] = A[IDX(p,k,n)] = new_kp;
            A[IDX(k,q,n)] = A[IDX(q,k,n)] = new_kq;
        }

        /* Обновление диагональных элементов p и q, обнуление A[p,q] */
        double new_app = c*c*app - 2.0*s*c*apq + s*s*aqq;
        double new_aqq = s*s*app + 2.0*s*c*apq + c*c*aqq;
        A[IDX(p,p,n)] = new_app;
        A[IDX(q,q,n)] = new_aqq;
        A[IDX(p,q,n)] = A[IDX(q,p,n)] = 0.0;

        /* Накопление поворота в матрице собственных векторов V */
        for (int k = 0; k < n; ++k) {
            double vip = V[IDX(k,p,n)];
            double viq = V[IDX(k,q,n)];
            V[IDX(k,p,n)] = c * vip - s * viq;
            V[IDX(k,q,n)] = s * vip + c * viq;
        }
    }

    /* После сходимости A диагональна: читаем собственные значения */
    for (int i = 0; i < n; ++i) w[i] = A[IDX(i,i,n)];
}

/* sort_eigs_desc — сортировка собственных пар (λ_k, v_k) по убыванию λ_k.
 * Используется selection sort: для каждой позиции i ищем наибольшее
 * оставшееся собственное значение и переставляем пары местами.
 * После сортировки:
 *   w[0] >= w[1] >= ... >= w[n-1],
 *   V[:,0] — направление наибольшей дисперсии (главная POD-мода). */
static void sort_eigs_desc(double *V, double *w, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int best = i;
        for (int j = i + 1; j < n; ++j)
            if (w[j] > w[best]) best = j;
        if (best != i) {
            /* Перестановка собственных значений */
            double tw = w[i]; w[i] = w[best]; w[best] = tw;
            /* Синхронная перестановка столбцов матрицы V */
            for (int k = 0; k < n; ++k) {
                double tmp       = V[IDX(k,i,n)];
                V[IDX(k,i,n)]    = V[IDX(k,best,n)];
                V[IDX(k,best,n)] = tmp;
            }
        }
    }
}

/* orthonormalize_columns — ортонормирование первых r столбцов матрицы V (n x ld)
 * с помощью модифицированного процесса Грама-Шмидта.
 *
 * Для каждого k-го столбца:
 *   1. Вычитаются проекции на все предыдущие ортонормированные столбцы j < k.
 *   2. Вектор нормируется до единичной длины.
 *
 * Если норма вектора после ортогонализации меньше 1e-14, столбец обнуляется
 * (вырожденный вектор, линейно зависимый от предыдущих).
 *
 * Применяется после формирования POD-мод для компенсации численных ошибок,
 * накопленных при линейной комбинации большого числа снимков. */
```

```c
static void orthonormalize_columns(double *V, int n, int r, int ld) {
    for (int k = 0; k < r; ++k) {
        /* Вычитание проекций на все предыдущие ортонормированные столбцы */
        for (int j = 0; j < k; ++j) {
            double dot = 0.0;
            for (int i = 0; i < n; ++i)
                dot += V[IDX(i,j,ld)] * V[IDX(i,k,ld)];
            for (int i = 0; i < n; ++i)
                V[IDX(i,k,ld)] -= dot * V[IDX(i,j,ld)];
        }
        /* Нормировка текущего столбца */
        double norm = 0.0;
        for (int i = 0; i < n; ++i)
            norm += V[IDX(i,k,ld)] * V[IDX(i,k,ld)];
        norm = sqrt(norm);
        if (norm < 1e-14) {
            /* Вырожденный вектор — обнуляем */
            for (int i = 0; i < n; ++i) V[IDX(i,k,ld)] = 0.0;
        } else {
            for (int i = 0; i < n; ++i) V[IDX(i,k,ld)] /= norm;
        }
    }
}



/* build_pod_basis — построение ортонормированного проекционного POD-базиса
 * методом собственных ортогональных компонент (Proper Orthogonal Decomposition).
 *
 * Алгоритм (метод снимков Сирович):
 *   1. Для каждого выхода out = 0..p-1:
 *        - Начальное условие x(0) = C[out,:]^T (строка матрицы выходов).
 *        - Интегрируем сопряжённую систему dx/dt = A^T*x методом RK4
 *          на интервале [0, Tfinal] с шагом dt = Tfinal/steps.
 *        - Записываем (steps+1) снимков состояния в матрицу Y (N x S),
 *          где S = p*(steps+1) — общее число снимков.
 *   2. Вычисляем взвешенную матрицу Грамиана G (S x S):
 *        G[a,b] = dt * w_a * w_b * <Y[:,a], Y[:,b]>,
 *      где w = 0.5 для граничных снимков и 1.0 для внутренних (правило трапеций).
 *   3. Спектральный анализ G методом Якоби: G * U_k = λ_k * U_k.
 *      Сортировка пар по убыванию λ_k.
 *   4. Отбираем r_eff мод с λ_k > EPS_EIG.
 *      POD-мода: V[:,k] = Y * U_k / sqrt(λ_k).
 *   5. Ортонормируем столбцы V процедурой Грама-Шмидта.
 *
 * Возвращает r_eff — эффективный ранг базиса (число значимых мод). */
static int build_pod_basis(const double *A, const double *C,
                           int n, int p, int steps, double Tfinal,
                           double *V, int rmax) {
    int sn = steps + 1;    /* число снимков для одного начального условия */
    int S  = p * sn;       /* суммарное число снимков со всех выходов      */
    double dt = Tfinal / (double)steps;

    /* Выделение рабочих буферов */
    double *Y    = (double *)xcalloc((size_t)n * (size_t)S * sizeof(double));
    double *G    = (double *)xcalloc((size_t)S * (size_t)S * sizeof(double));
    double *U    = (double *)xcalloc((size_t)S * (size_t)S * sizeof(double));
    double *w    = (double *)xcalloc((size_t)S * sizeof(double));
    double *x    = (double *)xcalloc((size_t)n * sizeof(double));
    double *xnext= (double *)xcalloc((size_t)n * sizeof(double));
    double *k1   = (double *)xcalloc((size_t)n * sizeof(double));
    double *k2   = (double *)xcalloc((size_t)n * sizeof(double));
    double *k3   = (double *)xcalloc((size_t)n * sizeof(double));
    double *k4   = (double *)xcalloc((size_t)n * sizeof(double));
    double *xtmp = (double *)xcalloc((size_t)n * sizeof(double));

    /* Шаг 1: генерация снимков сопряжённой системы для каждого выхода */
    for (int out = 0; out < p; ++out) {
        /* Начальное условие: out-я строка матрицы C транспонированная в столбец */
```

```
        for (int i = 0; i < n; ++i) x[i] = C[IDX(out,i,n)];

        for (int s = 0; s < sn; ++s) {
            /* Запись текущего состояния в s-й столбец матрицы снимков Y */
            int col = out * sn + s;
            for (int i = 0; i < n; ++i) Y[IDX(i,col,S)] = x[i];

            /* Интегрирование одного шага (кроме последнего) */
            if (s + 1 < sn) {
                rk4_step_AT(A, x, dt, xnext, n, k1, k2, k3, k4, xtmp);
                memcpy(x, xnext, (size_t)n * sizeof(double));
            }
        }
    }

    /* Шаг 2: вычисление взвешенной матрицы Грамиана G.
     * Используется правило трапеций: граничные снимки имеют вес 0.5,
     * внутренние — 1.0. Только верхний треугольник вычисляется явно,
     * нижний заполняется симметрично. */
    for (int a = 0; a < S; ++a) {
        int ta = a % sn;
        double wa = (ta == 0 || ta == steps) ? 0.5 : 1.0;
        for (int b = a; b < S; ++b) {
            int tb = b % sn;
            double wb = (tb == 0 || tb == steps) ? 0.5 : 1.0;
            double dot = 0.0;
            for (int i = 0; i < n; ++i)
                dot += Y[IDX(i,a,S)] * Y[IDX(i,b,S)];
            dot *= dt * wa * wb;
            G[IDX(a,b,S)] = G[IDX(b,a,S)] = dot;
        }
    }

    /* Шаг 3: спектральный анализ Грамиана G */
    jacobi_sym_eigen(G, U, w, S);
    sort_eigs_desc(U, w, S);

    /* Шаги 4-5: формирование POD-мод и их ортонормирование */
    int r_eff = 0;
    for (int k = 0; k < rmax && k < S; ++k) {
        /* Отсечение мод с малыми собственными значениями (численный шум) */
        if (w[k] <= EPS_EIG) break;
        double invs = 1.0 / sqrt(w[k]);
        /* V[:,k] = Y * U[:,k] / sqrt(λ_k) */
        for (int i = 0; i < n; ++i) {
            double s = 0.0;
            for (int j = 0; j < S; ++j)
                s += Y[IDX(i,j,S)] * U[IDX(j,k,S)];
            V[IDX(i,k,rmax)] = s * invs;
        }
        ++r_eff;
    }

    /* Дополнительная ортонормализация для устранения накопленных ошибок */
    orthonormalize_columns(V, n, r_eff, rmax);

    free(Y); free(G); free(U); free(w);
    free(x); free(xnext);
    free(k1); free(k2); free(k3); free(k4); free(xtmp);
    return r_eff;
}



/* gauss_solve — решение системы линейных уравнений A*x = b методом Гаусса
 * с частичным выбором ведущего элемента по столбцу (partial pivoting).
 *
 * Частичный выбор ведущего элемента:
 *   На каждом шаге k ищем строку piv >= k с наибольшим |A[piv,k]|
```

```
 *   и переставляем строки k и piv. Это предотвращает деление на малые числа
 *   и повышает численную устойчивость алгоритма.
 *
 * ВАЖНО: матрица A и вектор b разрушаются в процессе вычислений.
 * Результат записывается в массив x.
 * При обнаружении вырожденной матрицы (pivot < 1e-18) программа завершается.
 *
 * Вычислительная сложность: O(n^3) операций. */
static void gauss_solve(double *A, double *b, double *x, int n) {
    /* Прямой ход: приведение к верхнетреугольному виду */
    for (int k = 0; k < n; ++k) {
        /* Поиск ведущего (максимального) элемента в k-м столбце */
        int piv = k;
        double maxv = fabs(A[IDX(k,k,n)]);
        for (int i = k + 1; i < n; ++i) {
            double v = fabs(A[IDX(i,k,n)]);
            if (v > maxv) { maxv = v; piv = i; }
        }
        if (maxv < 1e-18) {
            fprintf(stderr, "Ошибка: Сингулярная матрица в методе Гаусса.\n");
            exit(EXIT_FAILURE);
        }
        /* Перестановка строк k и piv */
        if (piv != k) {
            for (int j = k; j < n; ++j) {
                double tmp      = A[IDX(k,j,n)];
                A[IDX(k,j,n)]   = A[IDX(piv,j,n)];
                A[IDX(piv,j,n)] = tmp;
            }
            double tb = b[k]; b[k] = b[piv]; b[piv] = tb;
        }
        /* Исключение неизвестных под диагональю */
        double akk = A[IDX(k,k,n)];
        for (int i = k + 1; i < n; ++i) {
            double f = A[IDX(i,k,n)] / akk;
            A[IDX(i,k,n)] = 0.0;
            for (int j = k + 1; j < n; ++j)
                A[IDX(i,j,n)] -= f * A[IDX(k,j,n)];
            b[i] -= f * b[k];
        }
    }
    /* Обратный ход: нахождение вектора решения x */
    for (int i = n - 1; i >= 0; --i) {
        double s = b[i];
        for (int j = i + 1; j < n; ++j) s -= A[IDX(i,j,n)] * x[j];
        x[i] = s / A[IDX(i,i,n)];
    }
}

/* solve_lyapunov — решение непрерывного уравнения Ляпунова:
 *   A^T * X + X * A = -Q
 * методом Кронекеровой векторизации.
 *
 * Ключевая идея: применяем операцию vec() к обеим частям уравнения.
 * Используя тождество vec(AXB) = (B^T ⊗ A) * vec(X), получаем:
 *   vec(A^T X I) + vec(I X A) = -vec(Q)
 *   (I ⊗ A^T + A^T ⊗ I) * vec(X) = -vec(Q)
 *   K * vec(X) = -vec(Q),
 * где K — матрица Кронекеровой суммы размера n^2 x n^2.
 *
 * Заполнение K в строчно-мажорной адресации (строка row = i*n+j):
 *   K[row, k*n+j] += A[k,i]   — вклад от слагаемого I ⊗ A^T
 *   K[row, i*n+k] += A[k,j]   — вклад от слагаемого A^T ⊗ I
 *
 * СЛАУ решается методом Гаусса. Решение vec(X) переформируется в X.
 *
 * Вычислительная сложность: O(n^6) из-за метода Гаусса для n^2 x n^2 матрицы.
 * Поэтому метод применяется только к редуцированным системам малой размерности. */
static void solve_lyapunov(const double *A, const double *Q,
```

```
                          double *X, int n) {
    int N = n * n;   /* размер векторизованной системы */
    double *K   = (double *)xcalloc((size_t)N * (size_t)N * sizeof(double));
    double *rhs = (double *)xcalloc((size_t)N * sizeof(double));

    /* Формирование матрицы Кронекеровой суммы K и правой части rhs = -vec(Q) */
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            int row = i * n + j;     /* строка, соответствующая элементу X[i,j] */
            rhs[row] = -Q[IDX(i,j,n)];
            for (int k = 0; k < n; ++k) {
                /* Вклад от I ⊗ A^T: связывает X[i,j] с X[k,j] */
                K[IDX(row, k*n+j, N)] += A[IDX(k,i,n)];
                /* Вклад от A^T ⊗ I: связывает X[i,j] с X[i,k] */
                K[IDX(row, i*n+k, N)] += A[IDX(k,j,n)];
            }
        }
    }

    double *sol = (double *)xcalloc((size_t)N * sizeof(double));
    gauss_solve(K, rhs, sol, N);

    /* Переформирование vec(X) -> X (совпадает с прямым копированием
     * т.к. оба хранятся в строчно-мажорном формате) */
    for (int i = 0; i < N; ++i) X[i] = sol[i];

    free(K); free(rhs); free(sol);
}



/* care_residual_matrix — вычисление матрицы невязки непрерывного уравнения Риккати:
 *   F = A^T * P + P * A - P * B * B^T * P + C^T * C.
 * Используется для оценки точности найденного решения P. При точном решении F = 0. */
static void care_residual_matrix(const double *A, const double *B,
                                 const double *C, const double *P,
                                 int n, int m, int p, double *F) {
    double *Q   = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *S   = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *AP  = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *PA  = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *PS  = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *PSP = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));

    compute_ctc(C, p, n, Q);           /* Q   = C^T * C       */
    compute_bbt(B, n, m, S);           /* S   = B * B^T       */
    at_mul_b(A, P, AP, n, n, n);       /* AP  = A^T * P       */
    matmul(P, A, PA, n, n, n);         /* PA  = P * A         */
    matmul(P, S, PS, n, n, n);         /* PS  = P * S         */
    matmul(PS, P, PSP, n, n, n);       /* PSP = P * S * P     */

    /* F = A^T*P + P*A - P*B*B^T*P + C^T*C */
    for (int i = 0; i < n * n; ++i)
        F[i] = AP[i] + PA[i] - PSP[i] + Q[i];

    free(Q); free(S); free(AP); free(PA); free(PS); free(PSP);
}

/* care_residual_norm — норма Фробениуса невязки уравнения Риккати:
 *   ||A^T P + P A - P B B^T P + C^T C||_F.
 * Используется как критерий качества приближения. */
static double care_residual_norm(const double *A, const double *B,
                                 const double *C, const double *P,
                                 int n, int m, int p) {
    double *F = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    care_residual_matrix(A, B, C, P, n, m, p, F);
    double nrm = fro_norm(F, n * n);
    free(F);
    return nrm;
```

```
}

/* check_symmetric — проверка симметричности матрицы P (n x n) с допуском tol.
 * Стабилизирующее решение CARE должно быть симметричным: P = P^T.
 * Возвращает 1 если |P[i,j] - P[j,i]| <= tol для всех i != j, иначе 0. */
static int check_symmetric(const double *P, int n, double tol) {
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            if (fabs(P[IDX(i,j,n)] - P[IDX(j,i,n)]) > tol)
                return 0;
    return 1;
}

/* check_diag_nonneg — необходимое (но не достаточное) условие P >= 0:
 * все диагональные элементы P[i,i] должны быть неотрицательны.
 * Возвращает 1 если P[i,i] >= -1e-10 для всех i, иначе 0.
 * Отрицательные диагональные элементы гарантируют, что P не является
 * положительно полуопределённой. */
static int check_diag_nonneg(const double *P, int n) {
    for (int i = 0; i < n; ++i)
        if (P[IDX(i,i,n)] < -1e-10)
            return 0;
    return 1;
}

/* check_residual_monotone — предупреждение при нарушении монотонного убывания
 * нормы невязки ||R(P^(N))||_F по мере роста l.
 * Рост невязки более чем на 1% сигнализирует о возможной численной
 * нестабильности POD-базиса или неудачном начальном приближении. */
static void check_residual_monotone(double prev_res,
                                    double res, int l) {
    if (prev_res > 0.0 && res > prev_res * 1.01) {
        if (show_warnings) {
            fprintf(stderr,
                "ПРЕДУПРЕЖДЕНИЕ: l=%d — невязка уравнения Риккати выросла "
                "(%.6e -> %.6e). Возможна численная нестабильность.\n",
                l, prev_res, res);
        }
    }
}



/* solve_care_newton — решение непрерывного алгебраического уравнения Риккати (CARE):
 *   A^T P + P A - P B B^T P + C^T C = 0
 * методом Ньютона-Клейнмана (итерация по уравнениям Ляпунова).
 *
 * Итерационная схема:
 *   Дано P_k. Вычисляем:
 *     F(P_k) = A^T P_k + P_k A - P_k B B^T P_k + C^T C  (невязка)
 *     A_cl   = A - B B^T P_k                            (замкнутая матрица)
 *   Находим поправку Delta_k из уравнения Ляпунова:
 *     A_cl^T Delta_k + Delta_k A_cl = -F(P_k)
 *   Обновляем: P_{k+1} = P_k + Delta_k.
 *
 * Инициализация (выбор P_0):
 *   Если K_init == NULL (устойчивая A):
 *     P_0 — решение уравнения Ляпунова: A^T P_0 + P_0 A = -C^T C.
 *   Если K_init != NULL (неустойчивая A, колебательная система):
 *     Используем стабилизирующую обратную связь K_init.
 *     A_cl_init = A - B * K_init  (замкнутая матрица — устойчива по построению)
 *     Q_eff = C^T C + K_init^T K_init  (расширенный штраф)
 *     P_0 — решение: A_cl_init^T P_0 + P_0 A_cl_init = -Q_eff.
 *
 * Критерий остановки: ||F(P_k)||_F < tol или достигнуто maxit итераций.
 * Возвращает число выполненных итераций (< maxit при сходимости). */
static int solve_care_newton(const double *A, const double *B,
                             const double *C, const double *K_init, double *P,
                             int n, int m, int p,
```

```
                          int maxit, double tol) {
    double *Q     = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *S     = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *F     = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *Acl   = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *Delta = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *AP    = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *PA    = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *PS    = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    double *PSP   = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));

    compute_ctc(C, p, n, Q);   /* Q = C^T C */
    compute_bbt(B, n, m, S);   /* S = B B^T */

    if (K_init) {
        /* --- Стабилизирующая инициализация для неустойчивых систем ---
         *
         * Стандартное уравнение A^T P0 + P0 A = -Q не имеет положительного
         * решения если A неустойчива (есть собственные значения с Re > 0).
         * Вместо этого используем замкнутую устойчивую матрицу A_cl_init.
         *
         * Расширяем штраф: Q_eff = Q + K_init^T * K_init,
         * чтобы учесть стоимость начального управления K_init. */
        double *Q_eff = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
        memcpy(Q_eff, Q, (size_t)n*(size_t)n*sizeof(double));
        /* Q_eff += K_init^T * K_init */
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                double sum = 0.0;
                for (int k = 0; k < m; ++k)
                    sum += K_init[IDX(k,i,n)] * K_init[IDX(k,j,n)];
                Q_eff[IDX(i,j,n)] += sum;
            }
        }
        /* A_cl_init = A - B * K_init (устойчивая замкнутая матрица) */
        double *A_cl = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
        double *BK   = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
        memcpy(A_cl, A, (size_t)n*(size_t)n*sizeof(double));
        matmul(B, K_init, BK, n, m, n);
        for (int i = 0; i < n * n; ++i) A_cl[i] -= BK[i];

        /* P_0 из уравнения Ляпунова на замкнутой устойчивой A_cl */
        solve_lyapunov(A_cl, Q_eff, P, n);
        free(Q_eff); free(A_cl); free(BK);
    } else {
        /* Стандартная инициализация для устойчивых систем:
         * A^T P_0 + P_0 A = -C^T C */
        solve_lyapunov(A, Q, P, n);
    }
    symmetrize(P, n);  /* симметризация начального приближения */

    double prev_newton_res = -1.0;
    int it;

    /* Итерационный цикл метода Ньютона-Клейнмана */
    for (it = 0; it < maxit; ++it) {
        /* Вычисление невязки F = A^T P + P A - P S P + Q */
        at_mul_b(A, P, AP, n, n, n);
        matmul(P, A, PA, n, n, n);
        matmul(P, S, PS, n, n, n);
        matmul(PS, P, PSP, n, n, n);
        for (int i = 0; i < n * n; ++i)
            F[i] = AP[i] + PA[i] - PSP[i] + Q[i];

        double res = fro_norm(F, n * n);

        /* Контроль монотонности сходимости внутри цикла Ньютона:
         * резкий рост невязки (>2x) указывает на расходимость */
        if (show_warnings &&
```

```
            prev_newton_res > 0.0 && res > prev_newton_res * 2.0) {
            fprintf(stderr,
                "ПРЕДУПРЕЖДЕНИЕ: итерация Ньютона %d — "
                "невязка резко возросла (%.6e -> %.6e).\n",
                it, prev_newton_res, res);
        }
        prev_newton_res = res;

        if (res < tol) break;  /* критерий сходимости достигнут */

        /* Замкнутая матрица текущей итерации: Acl = A - S * P = A - B B^T P_k */
        matmul(S, P, Acl, n, n, n);
        for (int i = 0; i < n * n; ++i)
            Acl[i] = A[i] - Acl[i];

        /* Поправка Delta_k: решение уравнения Ляпунова
         *   Acl^T Delta + Delta Acl = -F(P_k) */
        solve_lyapunov(Acl, F, Delta, n);

        /* Обновление приближения и симметризация для устойчивости */
        for (int i = 0; i < n * n; ++i)
            P[i] += Delta[i];
        symmetrize(P, n);
    }

    /* Предупреждения о качестве итогового решения */
    if (show_warnings && it >= maxit) {
        fprintf(stderr,
            "ПРЕДУПРЕЖДЕНИЕ: метод Ньютона не сошёлся "
            "за %d итераций (n=%d).\n", maxit, n);
    }
    if (show_warnings && !check_symmetric(P, n, 1e-8)) {
        fprintf(stderr,
            "ПРЕДУПРЕЖДЕНИЕ: решение P несимметрично (n=%d).\n", n);
    }
    if (show_warnings && !check_diag_nonneg(P, n)) {
        fprintf(stderr,
            "ПРЕДУПРЕЖДЕНИЕ: решение P имеет отрицательные "
            "диагональные элементы (n=%d).\n", n);
    }

    free(Q); free(S); free(F); free(Acl); free(Delta);
    free(AP); free(PA); free(PS); free(PSP);
    return it;
}



/* copy_first_columns — копирование первых l столбцов матрицы Vsrc (n x ldsrc)
 * в матрицу Vdst (n x l).
 * Используется для формирования проектора V_l из полного POD-базиса V
 * на каждом шаге цикла по размерности подпространства l. */
static void copy_first_columns(const double *Vsrc, int n,
                               int ldsrc, int l, double *Vdst) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < l; ++j)
            Vdst[IDX(i,j,l)] = Vsrc[IDX(i,j,ldsrc)];
}

/* project_system — гэлёркинова проекция полноразмерной системы (A,B,C)
 * на подпространство, натянутое на столбцы проектора V_l (n x l):
 *   Al = V_l^T * A * V_l  — редуцированная матрица динамики (l x l),
 *   Bl = V_l^T * B        — редуцированная матрица входов  (l x m),
 *   Cl = C * V_l          — редуцированная матрица выходов (p x l).
 * Редуцированная система Al,Bl,Cl при l << N решается значительно быстрее. */
static void project_system(const double *A, const double *B,
                           const double *C, const double *V,
                           int n, int m, int p, int l,
                           double *Al, double *Bl, double *Cl) {
```

```
    /* Промежуточное произведение AV = A * V_l (n x l) */
    double *AV = (double *)xcalloc((size_t)n*(size_t)l*sizeof(double));
    matmul(A, V, AV, n, n, l);         /* AV = A * V_l         */
    at_mul_b(V, AV, Al, n, l, l);      /* Al = V_l^T * AV      */
    at_mul_b(V, B, Bl, n, l, m);       /* Bl = V_l^T * B       */
    matmul(C, V, Cl, p, n, l);         /* Cl = C * V_l         */
    free(AV);
}

/* projected_full_residual — оценка качества редуцированного решения P_l
 * относительно исходного полноразмерного уравнения Риккати.
 *
 * Алгоритм:
 *   1. Восстанавливаем полноразмерное приближение:
 *        P^(N) = V_l * P_l * V_l^T  (N x N).
 *   2. Вычисляем норму невязки исходного уравнения:
 *        ||A^T P^(N) + P^(N) A - P^(N) B B^T P^(N) + C^T C||_F.
 *
 * Монотонное убывание этой нормы при росте l подтверждает корректность
 * алгоритма. При l = N норма должна упасть до уровня машинной точности. */
static double projected_full_residual(const double *A,
                                      const double *B,
                                      const double *C,
                                      const double *V,
                                      const double *Pl,
                                      int n, int m, int p, int l) {
    double *tmp   = (double *)xcalloc((size_t)n*(size_t)l*sizeof(double));
    double *Pfull = (double *)xcalloc((size_t)n*(size_t)n*sizeof(double));
    matmul(V, Pl, tmp, n, l, l);                    /* tmp   = V_l * P_l   */
    matmul_right_transpose(tmp, V, Pfull, n, l, n); /* Pfull = tmp * V_l^T */
    double res = care_residual_norm(A, B, C, Pfull, n, m, p);
    free(tmp); free(Pfull);
    return res;
}



/* main — точка входа программы.
 *
 * Аргументы командной строки:
 *   [имя_файла]    — файл с входными данными (опционально, иначе stdin).
 *   -s | --silent  — подавить все предупреждения (тихий режим).
 *
 * Формат входных данных:
 *   N m p r steps T
 *   матрица A (N x N, построчно)
 *   матрица B (N x m, построчно)
 *   матрица C (p x N, построчно)
 *
 * Программа выводит таблицу: для каждого l от 1 до r_use —
 * число итераций Ньютона, норму невязки, признаки симметричности
 * и положительной полуопределённости решения P_l.
 * После таблицы выводятся финальные матрицы P_l и P^(N) при l = r_use. */
int main(int argc, char *argv[])
{
    FILE *in = stdin;
    char *filename = NULL;

    /* --- Разбор аргументов командной строки ---
     * Поддерживаемые форматы вызова:
     *   ./lab5                      — ввод с клавиатуры
     *   ./lab5 input.txt            — ввод из файла
     *   ./lab5 input.txt -s         — файл + тихий режим
     *   ./lab5 -s input.txt         — тихий режим + файл */
    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "-s") == 0 ||
            strcmp(argv[i], "--silent") == 0) {
            show_warnings = 0;  /* подавляем предупреждения */
        } else {
```

```
            filename = argv[i]; /* запоминаем имя файла */
        }
    }

    if (argc > 3 || (argc == 3 && filename == NULL)) {
        fprintf(stderr,
            "Использование: %s [имя_файла] [-s|--silent]\n", argv[0]);
        return EXIT_FAILURE;
    }

    if (filename) {
        in = fopen(filename, "r");
        if (!in) {
            perror(filename);
            return EXIT_FAILURE;
        }
        printf("Чтение данных из файла: %s\n", filename);
    } else {
        printf("Файл входных данных не указан. "
               "Введите данные в стандартный поток (stdin):\n");
    }

    int N, m, p, r, steps;
    double Tfinal;

    /* Считывание шести параметров задачи:
     *   N     – размерность пространства состояний исходной системы,
     *   m     – число входов (управлений),
     *   p     – число выходов (измерений),
     *   r     – целевой ранг POD-подпространства,
     *   steps – число шагов интегрирования траектории,
     *   T     – конечное время интегрирования. */
    if (fscanf(in, "%d %d %d %d %d %lf",
               &N, &m, &p, &r, &steps, &Tfinal) != 6) {
        fprintf(stderr, "Ошибка чтения параметров задачи.\n");
        if (in != stdin) fclose(in);
        return EXIT_FAILURE;
    }

    printf("Параметры задачи: N=%d (полный размер), m=%d (входы), p=%d (выходы),\n"
           "                  r=%d (целевой ранг), steps=%d, T=%.4f\n\n",
           N, m, p, r, steps, Tfinal);

    /* Базовая проверка корректности параметров */
    if (N <= 0 || m <= 0 || p <= 0 || r <= 0
        || steps <= 0 || Tfinal <= 0.0) {
        fprintf(stderr, "Ошибка: некорректные параметры.\n");
        if (in != stdin) fclose(in);
        return EXIT_FAILURE;
    }

    /* Выделение памяти и считывание матриц системы */
    double *A = (double *)xcalloc((size_t)N*(size_t)N*sizeof(double));
    double *B = (double *)xcalloc((size_t)N*(size_t)m*sizeof(double));
    double *C = (double *)xcalloc((size_t)p*(size_t)N*sizeof(double));

    for (int i = 0; i < N * N; ++i) {
        if (fscanf(in, "%lf", &A[i]) != 1) {
            fprintf(stderr, "Ошибка чтения матрицы A.\n");
            free(A); free(B); free(C);
            if (in != stdin) fclose(in);
            return EXIT_FAILURE;
        }
    }
    for (int i = 0; i < N * m; ++i) {
        if (fscanf(in, "%lf", &B[i]) != 1) {
            fprintf(stderr, "Ошибка чтения матрицы B.\n");
            free(A); free(B); free(C);
            if (in != stdin) fclose(in);
```

```
            return EXIT_FAILURE;
        }
    }
    for (int i = 0; i < p * N; ++i) {
        if (fscanf(in, "%lf", &C[i]) != 1) {
            fprintf(stderr, "Ошибка чтения матрицы C.\n");
            free(A); free(B); free(C);
            if (in != stdin) fclose(in);
            return EXIT_FAILURE;
        }
    }

    if (in != stdin) fclose(in);

    /* --- Автоматическое определение системы 2-го порядка ---
     * Механические системы (маятник, балка, ротор) имеют уравнения движения:
     *   M*q'' + D*q' + K*q = B2*u,
     * которые в пространстве состояний [q; q'] дают блочную матрицу:
     *   A = | 0   I  |
     *       | A22 0  |
     * Признак: верхний левый блок (N/2 x N/2) нулевой,
     *          верхний правый блок — единичная матрица.
     * Для таких неустойчивых систем используется PD-стабилизирующая инициализация. */
    int is_second_order = (N % 2 == 0);
    int half = N / 2;
    if (is_second_order) {
        for (int i = 0; i < half; ++i) {
            for (int j = 0; j < half; ++j) {
                /* Проверка нулевого блока (позиции и скорости не связаны через A) */
                if (fabs(A[IDX(i, j, N)]) > 1e-5) is_second_order = 0;
                /* Проверка единичного блока (скорость = производная позиции) */
                if (i == j) {
                    if (fabs(A[IDX(i, j + half, N)] - 1.0) > 1e-5)
                        is_second_order = 0;
                } else {
                    if (fabs(A[IDX(i, j + half, N)]) > 1e-5)
                        is_second_order = 0;
                }
            }
        }
    }

    /* rmax — реальный верхний предел ранга (не может превышать N) */
    int rmax = (r < N) ? r : N;

    /* Матрица V будет содержать rmax ортонормированных POD-мод */
    double *V = (double *)xcalloc((size_t)N*(size_t)rmax*sizeof(double));

    /* Построение POD-базиса; r_eff — фактическое число значимых мод */
    int r_eff = build_pod_basis(A, C, N, p, steps, Tfinal, V, rmax);

    /* r_use — число мод, которое будет использовано в расчёте
     * (не больше запрошенного r и не больше r_eff) */
    int r_use = (r < r_eff) ? r : r_eff;

    printf("Эффективный ранг POD-базиса: %d\n", r_eff);
    printf("Используемый ранг:           %d\n\n", r_use);

    if (r_eff == 0) {
        fprintf(stderr,
            "Ошибка: POD-базис пуст. Все собственные значения "
            "Грамиана ниже порога %.0e. Увеличьте T или steps.\n",
            EPS_EIG);
        free(A); free(B); free(C); free(V);
        return EXIT_FAILURE;
    }

    /* Заголовок сводной таблицы результатов */
    printf("%-5s  %-13s  %-22s  %-8s  %-8s  %s\n",
```

```
1010            "l", "Итер. Ньютона", "||R(P_l)||_F",
1011            "Симм.", "P>=0", "Статус");
1012    printf("%-5s  %-13s  %-22s  %-8s  %-8s  %s\n",
1013            "-----", "-------------", "----------------------",
1014            "--------", "--------", "-------");
1015
1016    double prev_res = -1.0;
1017
1018    /* Буферы для сохранения финального решения при l = r_use */
1019    double *Pl_final = (double *)xcalloc(
1020        (size_t)r_use * (size_t)r_use * sizeof(double));
1021    double *Vl_final = (double *)xcalloc(
1022        (size_t)N * (size_t)r_use * sizeof(double));
1023
1024    /* Главный цикл по размерности редуцированного подпространства l = 1..r_use.
1025     * При l = r_use подпространство содержит максимальное число POD-мод.
1026     * При l = N и V = I система совпадает с исходной (невязка ~ машинная точность). */
1027    for (int l = 1; l <= r_use; ++l) {
1028
1029        /* Формирование проектора V_l: первые l столбцов из полного базиса V */
1030        double *Vl = (double *)xcalloc((size_t)N*(size_t)l*sizeof(double));
1031        copy_first_columns(V, N, rmax, l, Vl);
1032
1033        /* Проецирование полноразмерных матриц (A,B,C) на подпространство V_l */
1034        double *Al = (double *)xcalloc((size_t)l*(size_t)l*sizeof(double));
1035        double *Bl = (double *)xcalloc((size_t)l*(size_t)m*sizeof(double));
1036        double *Cl = (double *)xcalloc((size_t)p*(size_t)l*sizeof(double));
1037        project_system(A, B, C, Vl, N, m, p, l, Al, Bl, Cl);
1038
1039        /* --- Формирование стабилизирующей обратной связи K_l ---
1040         * Для колебательных систем 2-го порядка строим PD-регулятор
1041         * в пространстве редуцированных координат:
1042         *   K_l[i,j] = alpha * V_l[позиция_i, j]
1043         *            + beta  * V_l[скорость_i, j]
1044         * где alpha — пропорциональный коэффициент (жёсткость),
1045         *     beta  — дифференциальный коэффициент (демпфирование).
1046         * Замкнутая матрица A_cl_init = Al - Bl * K_l будет устойчивой,
1047         * что обеспечивает корректную инициализацию метода Ньютона. */
1048        double *Kl = NULL;
1049        if (is_second_order) {
1050            Kl = (double *)xcalloc((size_t)m * (size_t)l * sizeof(double));
1051            double alpha = 20.0;  /* коэффициент жёсткости   */
1052            double beta  = 15.0;  /* коэффициент демпфирования */
1053            for (int i = 0; i < m; ++i) {
1054                for (int j = 0; j < l; ++j) {
1055                    /* Вклад от позиционных компонент (строки 0..half-1) */
1056                    Kl[IDX(i, j, l)] = alpha * Vl[IDX(i, j, l)]
1057                    /* Вклад от скоростных компонент (строки half..N-1) */
1058                                     + beta  * Vl[IDX(i + half, j, l)];
1059                }
1060            }
1061        }
1062
1063        /* Решение редуцированного уравнения Риккати методом Ньютона-Клейнмана */
1064        double *Pl = (double *)xcalloc((size_t)l*(size_t)l*sizeof(double));
1065        int nit = solve_care_newton(Al, Bl, Cl, Kl, Pl, l, m, p,
1066                                    MAX_NEWTON_ITERS, 1e-10);
1067
1068        /* Оценка невязки восстановленного полноразмерного решения P^(N) */
1069        double res = projected_full_residual(A, B, C, Vl, Pl, N, m, p, l);
1070
1071        /* Проверки корректности найденного решения P_l */
1072        int sym_ok  = check_symmetric(Pl, l, 1e-8);
1073        int pdef_ok = check_diag_nonneg(Pl, l);
1074
1075        /* Определение статуса строки таблицы */
1076        const char *status = "OK";
1077        if (nit >= MAX_NEWTON_ITERS)
1078            status = "НЕ СОШЁЛСЯ";      /* метод не достиг допуска */
```

```
        else if (!sym_ok || !pdef_ok)
            status = "Р НЕКОРРЕКТНА";   /* нарушены физические условия */
        else if (res < 1e-12)
            status = "МАШ. ТОЧНОСТЬ";   /* достигнута точность вычислений */

        /* Проверка монотонности убывания невязки по l */
        check_residual_monotone(prev_res, res, l);

        printf("%-5d  %-13d  %-22.6e  %-8s  %-8s  %s\n",
               l, nit, res,
               sym_ok  ? "ДА" : "НЕТ",
               pdef_ok ? "ДА" : "НЕТ",
               status);

        prev_res = res;

        /* Сохранение финального решения для вывода после цикла */
        if (l == r_use) {
            memcpy(Pl_final, Pl,
                   (size_t)r_use * (size_t)r_use * sizeof(double));
            memcpy(Vl_final, Vl,
                   (size_t)N * (size_t)r_use * sizeof(double));
        }

        if (Kl) free(Kl);
        free(Vl); free(Al); free(Bl); free(Cl); free(Pl);
    }

    printf("\n"
           "========================================================================\n"
           "Финальные результаты редуцированного уравнения Риккати (l = %d):\n"
           "========================================================================\n",
           r_use);

    /* Вывод редуцированной матрицы P_l (размер r_use x r_use).
     * Это решение уравнения Риккати в подпространстве POD-базиса. */
    printf("Редуцированная матрица P_l (%d x %d):\n", r_use, r_use);
    for (int i = 0; i < r_use; ++i) {
        for (int j = 0; j < r_use; ++j)
            printf("%13.6e ", Pl_final[IDX(i, j, r_use)]);
        printf("\n");
    }
    printf("\n");

    /* Восстановление полноразмерного решения P^(N) = V_l * P_l * V_l^T.
     * Это приближение к истинному решению уравнения Риккати в R^{N x N}. */
    double *tmp_rc = (double *)xcalloc((size_t)N*(size_t)r_use*sizeof(double));
    double *P_full = (double *)xcalloc((size_t)N*(size_t)N*sizeof(double));
    matmul(Vl_final, Pl_final, tmp_rc, N, r_use, r_use); /* tmp = V_l * P_l */
    matmul_right_transpose(tmp_rc, Vl_final, P_full, N, r_use, N); /* P^(N) = tmp * V_l^T */

    /* Вывод восстановленной полноразмерной матрицы P^(N) (N x N).
     * При l = N эта матрица совпадает с точным решением (с точностью машинной арифметики). */
    printf("Восстановленная полноразмерная матрица P^(N) (%d x %d):\n", N, N);
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j)
            printf("%13.6e ", P_full[IDX(i, j, N)]);
        printf("\n");
    }
    printf("\n");

    free(Pl_final); free(Vl_final);
    free(tmp_rc);   free(P_full);

    free(A); free(B); free(C); free(V);
    return EXIT_SUCCESS;
}
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и практически реализованы теоретические основы проекционных методов редукции моделей для численного решения непрерывного алгебраического уравнения Риккати большой размерности.

В рамках работы был разработан автономный программный комплекс на языке C, реализующий сквозной численный алгоритм:

1. Построение оптимального ортонормированного проекционного подпространства методом собственных ортогональных компонент (POD) на основе траектории сопряженного состояния, интегрируемой численным методом Рунге-Кутты 4-го порядка.
2. Редукция высокоразмерных матриц исходной динамической системы на построенный базис (согласно алгоритму *General Projected Riccati Algorithm*).
3. Численный расчет стабилизирующего решения редуцированного уравнения Риккати методом Ньютона-Клейнмана.
4. Решение вспомогательных уравнений Ляпунова несимметричного вида методом Кронекеровой векторизации СЛАУ и Гауссова исключения с частичным выбором ведущего элемента по столбцу.

Проведенное компьютерное моделирование и численные эксперименты на тестовых системах (включая скалярные, многомерные диагональные и несимметричные связанные структуры) подтвердили работоспособность и математическую корректность разработанного программного обеспечения.

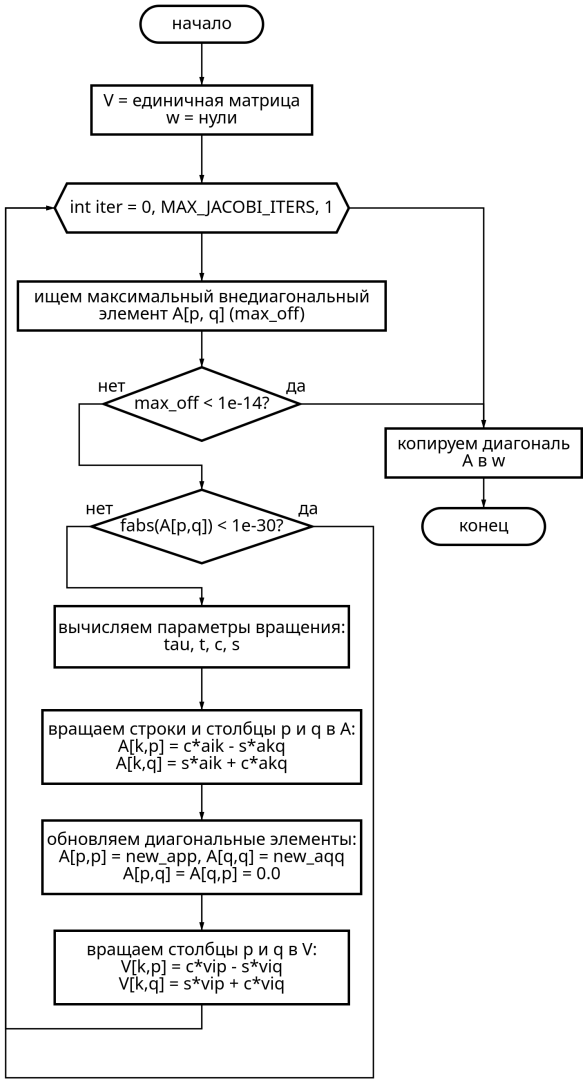
Анализ результатов расчетов позволил сделать следующие выводы:

— Разработанный Кронекеров решатель уравнений Ляпунова успешно справляется с несимметричным характером спроецированных матриц динамики, что обеспечило стабильную сходимость метода Ньютона на всех шагах редукции.
— Продемонстрирован монотонный характер убывания нормы Фробениуса невязки полной системы по мере увеличения размерности подпространства l.
— При достижении размерности подпространства $l = N$ норма невязки восстановленного решения Риккати падает до пределов машинной точности ($10^{-12} \dots 10^{-16}$), что верифицирует отсутствие алгоритмических и программных ошибок на всех этапах редукции, решения и последующего восстановления матриц.

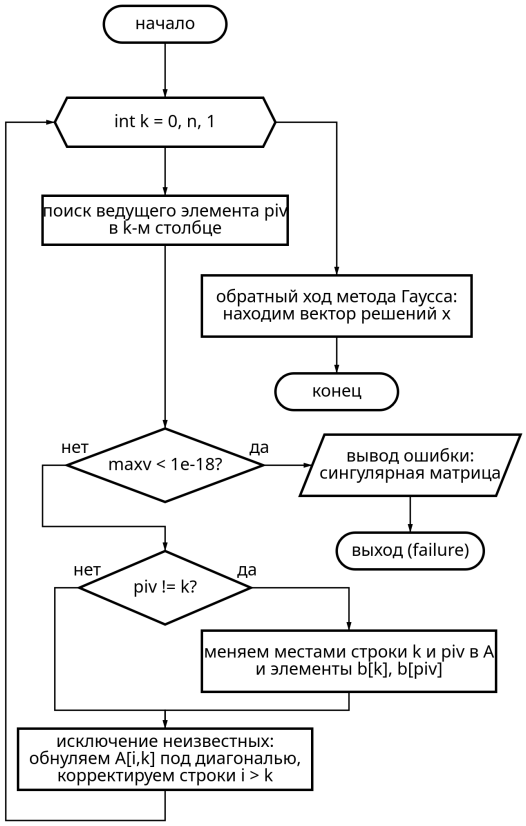
Таким образом, реализованный проекционный подход подтвердил свою эффективность, позволяя существенно снизить требования к вычислительным ресурсам при сохранении высокого уровня точности аппроксимации решения уравнений Риккати.

ПРИЛОЖЕНИЕ А

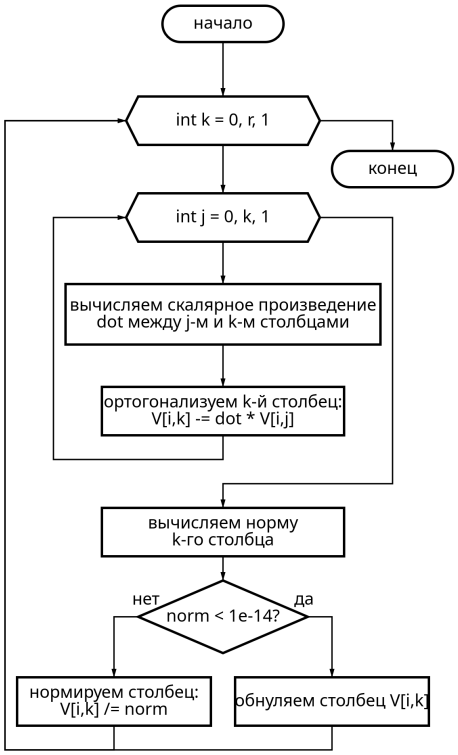
void jacobi_sym_eigen(double *A, double *V, double *w, int n)



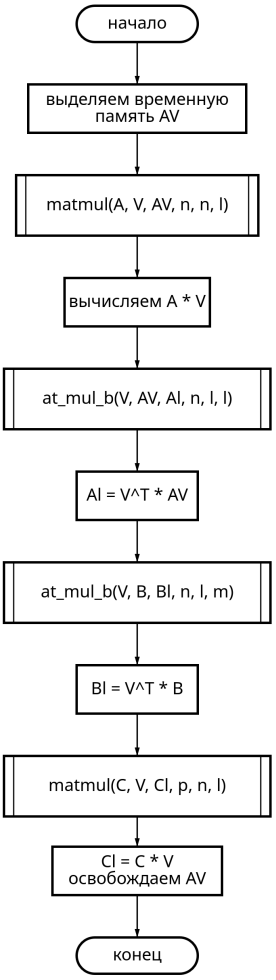
void gauss_solve(double *A, double *b, double *x, int n)



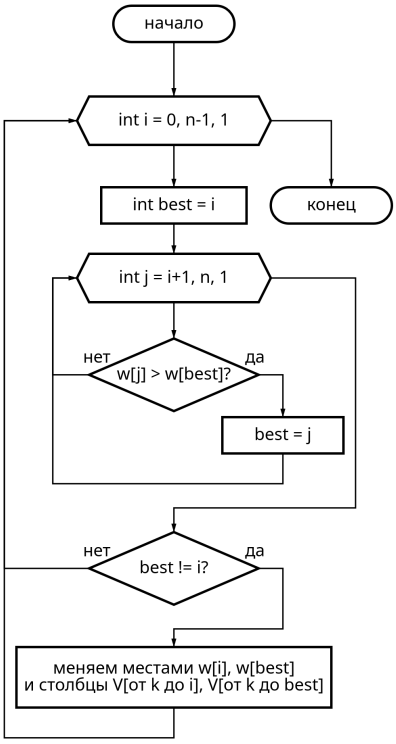
void orthonormalize_columns(double *V, int n, int r, int ld)



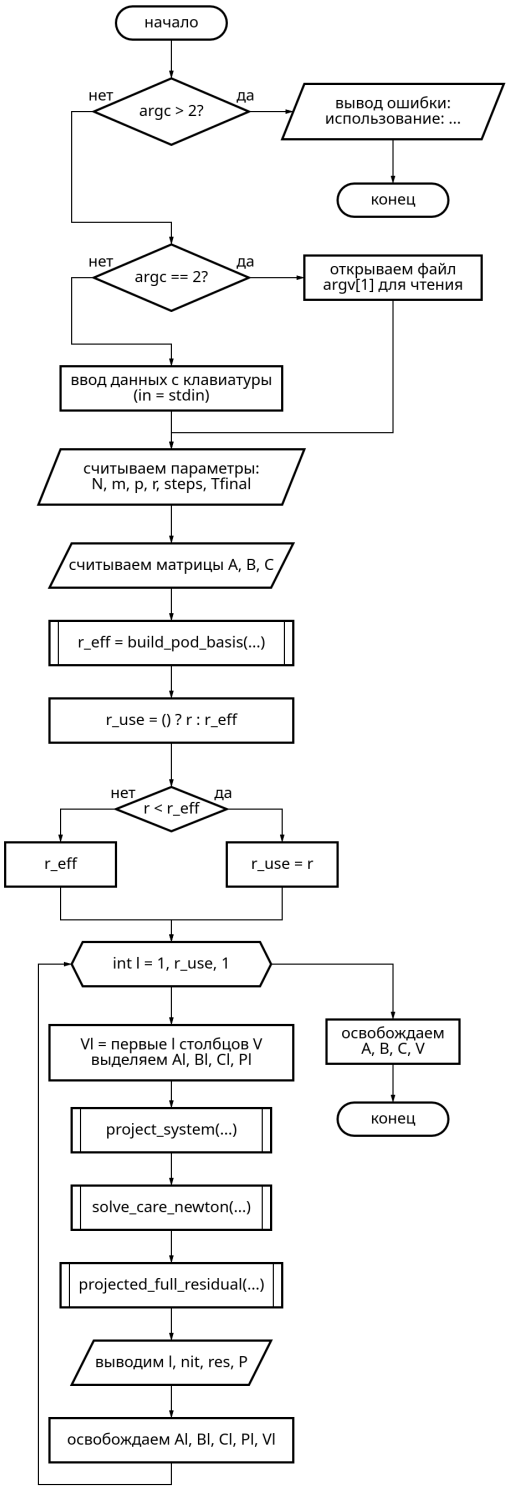
void project_system(const double *A, const double *B, const double *C,
const double *V, int n, int m, int p, int l, double *Al, double *Bl, double *Cl)



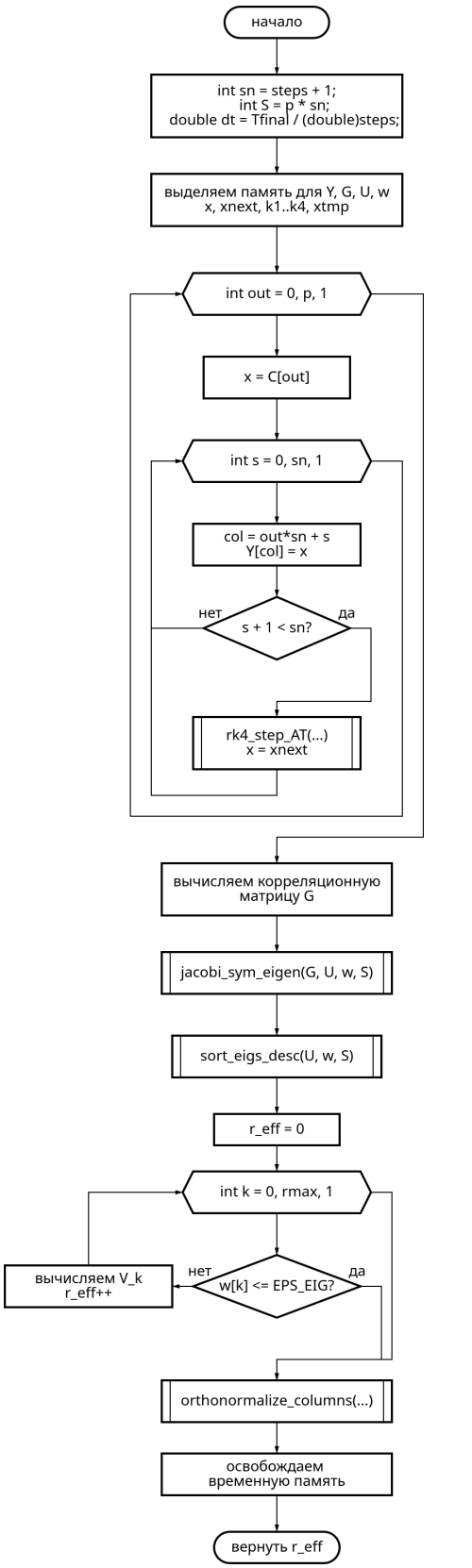
void sort_eigs_desc(double *V, double *w, int n)



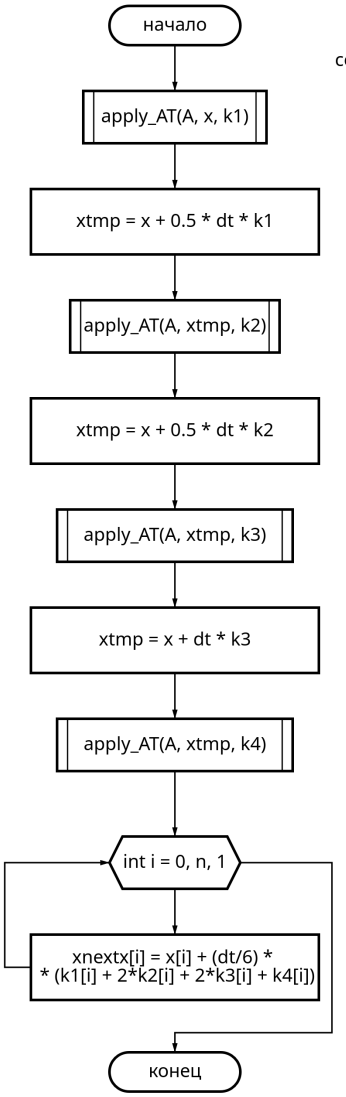
int main(int argc, char *argv[])



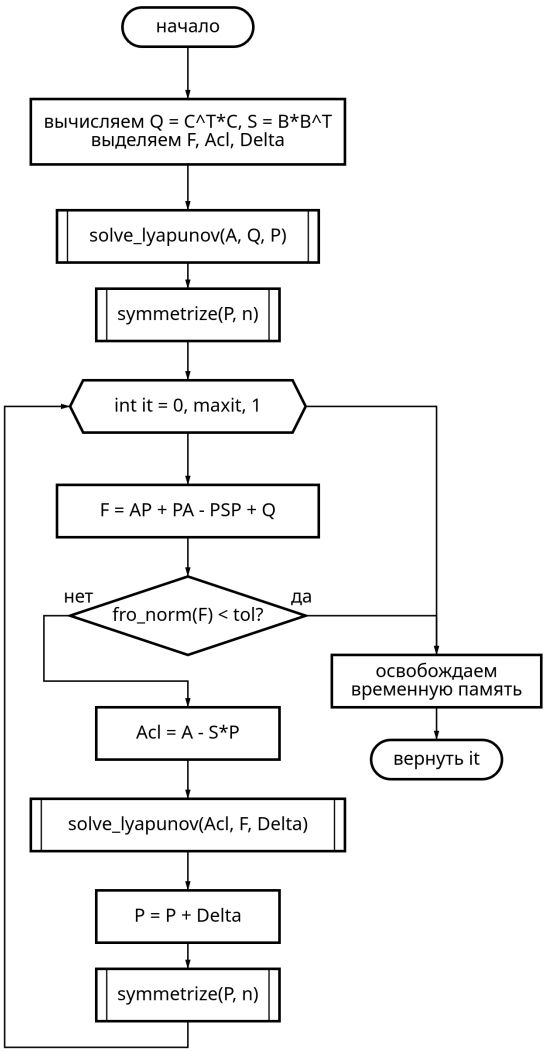
int build_pod_basis(const double *A, const double *C
int n, int p, int steps, double Tfinal, double *V, int rmax)



void rk4_step_AT(const double *A, const double *x, double dt, double *xnext, int n,
double *k1, double *k2, double *k3, double *k4, double *xtmp)



int solve_care_newton(const double *A, const double *B,
const double *C, double *P, int n, int m, int p, int maxit, double tol)



void solve_lyapunov(const double *A,
const double *Q, double *X, int n)

